In [1]:
```
# importing pandas for reading data and performing other dataframe related c
import pandas as pd

# importing numpy for performing various numerical operations
import numpy as np

# importing matplotlib and seaborn for visualization
import matplotlib.pyplot as plt
import seaborn as sns

# importing the historical banking data as a pandas DataFrame
historical_df = pd.read_csv('Historical_Banking_Data.csv', low_memory=False)

# creating a copy of the data for further analysis
df = historical_df.copy()
```

In [2]:
```
df.head()
```

Out[2]:

	ID	Customer_ID	Month	Name	Age	SSN	Occupation	Annual_Inc
0	0x1602	CUS_0xd40	January	Aaron Maashoh	23	821-00-0265	Scientist	191
1	0x1603	CUS_0xd40	February	Aaron Maashoh	23	821-00-0265	Scientist	191
2	0x1604	CUS_0xd40	March	Aaron Maashoh	-500	821-00-0265	Scientist	191
3	0x1605	CUS_0xd40	April	Aaron Maashoh	23	821-00-0265	Scientist	191
4	0x1606	CUS_0xd40	May	Aaron Maashoh	23	821-00-0265	Scientist	191

5 rows × 28 columns

In [3]:
```
print("The number of rows are {} and the number of columns are {}".format(df
```

```
The number of rows are 100000 and the number of columns are 28
```

In [4]:
```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 100000 entries, 0 to 99999
Data columns (total 28 columns):
 #   Column                    Non-Null Count   Dtype  
---  ------                    --------------   -----  
 0   ID                        100000 non-null  object 
 1   Customer_ID               100000 non-null  object 
 2   Month                     100000 non-null  object 
 3   Name                      90015 non-null   object 
 4   Age                       100000 non-null  object 
 5   SSN                       100000 non-null  object 
 6   Occupation                100000 non-null  object 
 7   Annual_Income             100000 non-null  object 
 8   Monthly_Inhand_Salary     84998 non-null   float64
 9   Num_Bank_Accounts         100000 non-null  int64  
 10  Num_Credit_Card           100000 non-null  int64  
 11  Interest_Rate             100000 non-null  int64  
 12  Num_of_Loan               100000 non-null  object 
 13  Type_of_Loan              88592 non-null   object 
 14  Delay_from_due_date       100000 non-null  int64  
 15  Num_of_Delayed_Payment    92998 non-null   object 
 16  Changed_Credit_Limit      100000 non-null  object 
 17  Num_Credit_Inquiries      98035 non-null   float64
 18  Credit_Mix                100000 non-null  object 
 19  Outstanding_Debt          100000 non-null  object 
 20  Credit_Utilization_Ratio  100000 non-null  float64
 21  Credit_History_Age        90970 non-null   object 
 22  Payment_of_Min_Amount     100000 non-null  object 
 23  Total_EMI_per_month       100000 non-null  float64
 24  Amount_invested_monthly   95521 non-null   object 
 25  Payment_Behaviour         100000 non-null  object 
 26  Monthly_Balance           98800 non-null   object 
 27  Credit_Score              100000 non-null  object 
dtypes: float64(4), int64(4), object(20)
memory usage: 21.4+ MB
```

In [5]:
```
df.describe().T
```

Out[5]:

	count	mean	std	min	
Monthly_Inhand_Salary	84998.0	4194.170850	3183.686167	303.645417	1625.5
Num_Bank_Accounts	100000.0	17.091280	117.404834	-1.000000	3.0
Num_Credit_Card	100000.0	22.474430	129.057410	0.000000	4.0
Interest_Rate	100000.0	72.466040	466.422621	1.000000	8.0
Delay_from_due_date	100000.0	21.068780	14.860104	-5.000000	10.0
Num_Credit_Inquiries	98035.0	27.754251	193.177339	0.000000	3.0
Credit_Utilization_Ratio	100000.0	32.285173	5.116875	20.000000	28.0
Total_EMI_per_month	100000.0	1403.118217	8306.041270	0.000000	30.3

In [6]:
```
df.describe(exclude=np.number).T
```

Out[6]:

	count	unique	top
ID	100000	100000	0x1602
Customer_ID	100000	12500	CUS_0xd40
Month	100000	8	January
Name	90015	10139	Langep
Age	100000	1788	38
SSN	100000	12501	#F%$D@*&8
Occupation	100000	16	_______
Annual_Income	100000	18940	36585.12
Num_of_Loan	100000	434	3
Type_of_Loan	88592	6260	Not Specified
Num_of_Delayed_Payment	92998	749	19
Changed_Credit_Limit	100000	4384	_
Credit_Mix	100000	4	Standard
Outstanding_Debt	100000	13178	1360.45
Credit_History_Age	90970	404	15 Years and 11 Months
Payment_of_Min_Amount	100000	3	Yes
Amount_invested_monthly	95521	91049	__10000__
Payment_Behaviour	100000	7	Low_spent_Small_value_payments
Monthly_Balance	98800	98792	__-333333333333333333333333333__
Credit_Score	100000	3	Standard

In [7]:
```
df.Month.value_counts()
```

Out[7]:
```
Month
January     12500
February    12500
March       12500
April       12500
May         12500
June        12500
July        12500
August      12500
Name: count, dtype: int64
```

In [8]:
```
df_without_na = df.dropna().copy()
```

In [9]:
```
for i in df_without_na:
 print('\n', i, df_without_na[i].unique())
```

```
 ID ['0x1602' '0x1608' '0x160e' ... '0x25fea' '0x25feb' '0x25fed']

 Customer_ID ['CUS_0xd40' 'CUS_0x21b1' 'CUS_0x2dbc' ... 'CUS_0xaf61' 'CUS_0x
8600'
 'CUS_0x942c']

 Month ['January' 'July' 'February' 'March' 'May' 'June' 'August' 'April']

 Name ['Aaron Maashoh' 'Rick Rothackerj' 'Langep' ... 'Chris Wickhamm'
 'Sarah McBridec' 'Nicks']

 Age ['23' '28_' '28' ... '8425' '2263' '1342']

 SSN ['821-00-0265' '004-07-5839' '486-85-3974' ... '133-16-7738' '031-35-09
42'
 '078-73-5990']

 Occupation ['Scientist' '_______' 'Teacher' 'Engineer' 'Entrepreneur' 'Lawy
er'
 'Media_Manager' 'Doctor' 'Journalist' 'Manager' 'Mechanic' 'Writer'
 'Accountant' 'Architect' 'Musician' 'Developer']

 Annual_Income ['19114.12' '34847.84' '34847.84_' ... '20002.88' '39628.99'
'39628.99_']

 Monthly_Inhand_Salary [ 1824.84333333  3037.98666667 12187.22       ...  30
97.00833333
  1929.90666667  3359.41583333]

 Num_Bank_Accounts [   3    2    1    0    8    5    6    7    9   10    4
67  528 1647
 1696  649  889 1620 1388 1429 1777 1096  803  494 1414  831  121  823
 1356 1651  711  210 1671  648  672 1662 1495  666  429  980  718 1312
  501  628 1016 1265  791  427 1036  619  555 1769  280  752  812 1487
 1019 1222  610 1714  525  797 1577  521  703  933  959  809 1089 1668
 1789  434 1295  677  120  698 1101  464 1462   45  244 1266  897  484
  331  826 1026  549  473 1628  371  445 1168  868  687 1003  598  351
 1342 1557  373  942  386 1448 1332 1033 1731 1214 1481  347  148 1467
 1114 1000  632  689  482 1303 1358  179  844 1140  732 1436   28  818
  731  534 1255 1763 1418  264  194  611 1065  584  550 1014  903 1748
   31  417   37 1424 1061  488 1274  526 1564  497 1135  441   74  314
 1408  475 1240 1657 1091 1264 1632 1562 1217 1391 1624 1794  397 1239
  847  786 1317   -1  302  456  198  122 1478 1547 1666 1241  275  566
  834 1174 1040  530  112  688  246 1600  483 1501  950 1211  129  540
 1252  499  702 1652  795  938  833 1654 1793  160 1491  423  928  339
  243 1756 1695  274  430 1247  490  726   42 1470 1739  887  385 1221
  406 1677 1567  182 1079  137 1124 1048 1069  589 1634   70 1153 1511
 1576 1337 1766  186  170  616   99  288  974 1595 1594  865  882  448
  875 1733   79  135  142 1461  560 1551  409  124  313 1363  232 1432
 1479 1407 1080 1719  158  563  312 1638  455 1676 1060  684  769 1423
 1502  936  350 1560 1013 1691  715 1570 1751 1194  411  308  796  912
 1256 1774 1181  837 1430   53 1447 1784  328  353  822  654  829  609
  166  845  172  808  270  710 1072  514 1331 1630 1616   50 1355  394
 1393  368 1018  229  265  894 1730 1076  449 1318 1155   30  817  709
 1670 1641 1782  119 1574 1137  947  851 1041 1257 1583 1323  575  196
   33  230  352 1250   39  683  511  918  734  316  771  705  105 1553
```

```
 424  493 1530 1604 1599 1747  435  782 1606 1622 1275 1771 1798  420
 832  932  468  442 1442  927  921 1398 1350  676 1426 1786  850  777
 375  318 1325  290 1352 1775 1365  848   84 1458  582 1261 1067  145
1139  471  967  539  858  701  354  162  467 1297  157  993  125  271
1669  272  564  997  333  714 1276  805 1320  245  446  978 1002 1631
1307 1378 1328 1309  505  197  999 1100   77 1506   11  218  392 1190
1321 1306   34  443  811  774  801  217  358  864  140 1361  481  512
1117  300 1263 1581 1298  330  310  637  758 1734 1288  326 1678 1444
1344 1166  828 1589 1637 1529 1627  115 1047 1617  480 1404  940 1305
1049 1645 1466  175 1165  581  857  854  725 1134  239  955 1381 1760
 360 1735  969  926 1281  591  708  784 1349  839 1039 1483 1395 1249
1207 1213 1219 1345  949  103  992 1314  151  624 1489  695  305  444
1077 1354  690 1389  968  856   72  415  971 1078   64 1701  546 1443
  27 1453 1709  304  991  193 1528  957  346 1416  396 1182  645  680
1703  472  463 1083   75  916  296  813  474  697]

 Num_Credit_Card [   4 1385    5    1    7    6 1029    8 1381    3    9 132
7   10    2
 943  262  809  932  688 1163 1315   11 1201  169 1423 1297  538 1191
 496  803  102  986 1426  211  615   97 1387  447  158 1141 1052  331
 996  912  419 1038  132   39  403  142  489  637 1354 1434  540  732
1208 1389 1480 1325  353  223 1084 1375  980 1323  889  297  924  773
1080 1168  363  397 1436  183 1116  117 1192  274  292  997  953  716
  86  507  648  736  634  309 1331 1165 1435  626 1414  151  490  909
 448  550 1127  121  762  751  457 1178  380  792  333  123  445  731
1174  450  329  584 1479 1032  995 1087  874  794  685  390  630  207
 172  156 1298   60 1356  527    0 1093 1010  501  200  681  663 1452
1286  799  851 1066 1009  106   25  766  829  431   66 1169  698 1135
1291 1233  616 1115  667  926  553  777  221  235  249  978  302  458
 982  863  174  365  555  710 1403  422 1395  929 1357  218  409  756
  83  236  215  880 1374  761  655  958  875  994  558   46  330  860
 816  240  976 1303  370  992  118 1202 1189  193 1336  734   77 1314
 257  703 1302  806  878  629  116  129  859  665 1105  707  633  311
1005 1498 1136  963  502  362  591 1181  554  993  578  133  176  824
1459   72  826  928  464 1308 1062 1143  143 1046  391 1229   98 1025
1170  270  170  598 1252  147  428  494 1392  676  841  805  702  552
 104  579  434 1205  911 1061  314 1316  962  449  446  296  456 1264
1223  757  631  564  583  854  972  745 1112 1341  305 1499 1320  954
1207  337 1391  857  467 1031 1083 1465  800 1180 1030  602  493  678
 444  187 1203  275  759 1332 1071  846 1090  105 1043  870  913  738
 155 1352 1076 1187 1494 1160  775  974  171  125  451 1019 1250  328
 316 1245  779  345 1213  487   61  523  965 1420 1292 1123  856  747
1156 1334 1217  947  733 1172  208  417   51  948  728  522   31  323
1413  219  690 1290  534  260  542  222 1379 1130  899 1117 1173 1186
 742  937   37 1419  704  477 1446 1239  739  100 1346 1321  933  559
 842 1485  242   28 1132  705 1002  574  148 1318 1394 1258  740 1221
 657 1249   70  791  838  248  440  247  374 1148  825 1493 1240  700
 204  680 1017  782  175 1114  284  360 1204  652  810 1069  251 1262
1409  670  781  566 1131  470 1450 1142  970  961  253  537  394 1242
 664 1454 1377  356 1100  510 1232  157 1338 1256 1210 1472 1155  570
  38 1007 1265 1351 1274  520 1108  146  793 1301 1271   29 1455  531
1299  662   22  153  398  381  238  964  892 1024  951  787   65 1056
1424 1104 1150 1074  654 1230  573 1312  301 1349 1199  772 1406  294
 656  557  619  303  896  348 1489  990 1496  891   15  668  764   52
 577  697   85 1473 1333  743  336 1124  817 1042  190 1492   48   92
 126  910  946  327  725  718  258  774 1399  159 1012  618   44  188
```

```
 1049 1326  483 1270  308  915 1198  760  795   27  245 1120  271 1453
  165  798 1044 1106  307  300  411  758 1386  475  379 1133 1407 1218
 1311  246  373  849  904  173  639  321 1269 1460  136  291  322  744
  231 1246  627  124 1048  478 1365   87 1458 1060 1343  684  906  925
 1037   49 1179  335 1255  603  167 1484 1231 1015  536 1008   71 1103
 1340  741  887  150  545  192  230  813  836  424  115  691  864  565
  180  850  675  179 1477  722   54 1216   69 1353 1289  352  197  184
  460  226   18 1089  141   33 1151  621  696  286 1023  144  581 1416
 1206  852 1243  430 1263  858  827  628   80 1047  917 1463  876  600
 1050  210 1226  412 1461 1412  543  498 1475 1421 1054  313  918 1102
 1247 1162  607 1193  811  848 1184  392  898 1070 1364  213 1079   16
 1440  837 1177  746 1380  749  802  687  541 1417  466 1478 1490  642
  625   21 1466  383  567  280  944  717 1195  243 1295 1248  202  462
 1129  830  259 1313   34  632  387  212  227  349 1073  209 1443 1283
 1064  659  516  695  660  867  304  346  107  138  692 1411  769  966
  220  834  399  340  479 1428  835  894 1376  112  154  350 1222  549
 1175 1486   43  298  191   96  119  101  130 1101  177  942  563  492
  689  386  195  341  279  789  723  344  737  635  418   89  320  535
  461  551  784  673  599  881  214  332  261  884  571  375  931  237
 1212 1328  384 1373 1260 1371  755  319  505  622  895  306 1157  776
  661  709  481  343  797   62  983 1013 1294  110  135 1107 1305  869
  991 1176 1065 1402  163  814  753  389  977  443  650  686 1430  679]

 Interest_Rate [   3    6    8 ... 1347  360 5729]

 Num_of_Loan ['4' '1' '3' '967' '-100' '2' '3_' '7' '5' '5_' '6' '8' '2_'
'9' '4_' '7_'
 '1_' '8_' '6_' '622' '9_' '472' '1017' '146' '563' '444' '720' '1485'
 '49' '737' '1106' '466' '728' '843' '597_' '617' '119' '663' '92_' '1019'
 '39' '848' '931' '1214' '424' '1001' '1152' '457' '1433' '1187' '52'
 '1047' '33' '699' '329' '1464' '132' '649' '545' '684' '1135' '1204' '58'
 '1406' '1348' '1461' '1312' '1424' '95' '1228' '1006' '795' '1185_'
 '1465' '1181' '70' '55' '1096' '904' '89' '1259' '527' '449' '418' '319'
 '23' '638' '138' '1480' '235_' '280' '274' '1459_' '404' '1354' '1391'
 '601' '1313' '898' '231' '752' '174' '834' '284' '438' '288' '1463' '719'
 '1015' '392' '1320_' '630_' '241' '31' '1217' '1030' '137' '164' '1088'
 '777' '613' '321' '661' '939' '562' '943' '955' '1318' '936' '430' '860'
 '191' '282' '757' '833' '292' '510' '332' '996' '311' '492' '123' '540'
 '131_' '1311_' '895' '1129_' '1014' '365' '742' '462' '832' '1225' '1412'
 '785_' '1127' '251' '101' '87' '659' '633' '387' '447' '773' '289' '143_'
 '1359' '1189' '1294' '201' '579' '814' '141' '1171_' '295' '1040' '1054'
 '1023' '1077' '1457' '1150' '701' '1209' '868' '19' '1297' '227_' '809'
 '1216' '520' '1274' '733' '991' '316' '926' '65' '875' '501' '1271' '636'
 '228' '1371' '254' '464' '1160' '1289' '1298' '799' '182' '574' '869'
 '54' '656' '778' '147' '1384' '275' '497' '927' '653' '662' '217' '529'
 '697' '1039' '227' '1345' '1279' '300' '1103' '504' '1365' '1400' '78'
 '1091' '344' '84' '1470' '1196' '1241' '1307' '1008' '917' '56' '41'
 '192' '978' '349' '966']

 Type_of_Loan ['Auto Loan, Credit-Builder Loan, Personal Loan, and Home Equi
ty Loan'
 'Credit-Builder Loan' 'Auto Loan, Auto Loan, and Not Specified' ...
 'Home Equity Loan, Auto Loan, Auto Loan, and Auto Loan'
 'Payday Loan, Student Loan, Mortgage Loan, and Not Specified'
 'Personal Loan, Auto Loan, Mortgage Loan, Student Loan, and Student Loan']
```

```
 Delay_from_due_date [ 3  7  5 10  8  0  4 -1 30 34 14 11 13  2 -2  1 16 17
23 22 15 12 51 53
 48 43 52 25 20 49 28 61 31 26 29 18 45  6 27 56 59 55 57 54 62 67 50 36
 41 19 21 24 65 32  9 39 47 46 60 64 33 35 44 38 58 63 42 40 37 -3 -5 -4
 66]

 Num_of_Delayed_Payment ['7' '8_' '4' '1' '-1' '0' '8' '5' '6' '3' '9' '2'
'14' '11' '20' '22'
 '10' '13' '13_' '14_' '16' '12' '12_' '18' '19' '17' '23' '24' '21' '15'
 '3083' '22_' '1338' '26' '11_' '25' '183_' '9_' '1106' '834' '19_' '24_'
 '3_' '23_' '2672' '20_' '2008' '538' '6_' '1_' '27' '-2' '3478' '2420'
 '15_' '707' '18_' '28' '5_' '4_' '16_' '1463' '-3' '4126' '17_' '2882'
 '1941' '7_' '2655' '3069' '306' '0_' '3539' '3684' '10_' '1823' '1946'
 '2297' '904' '929' '3568' '1552' '1697' '851' '1668' '808' '2689' '3858'
 '642' '3037' '3103' '2_' '1063' '2056' '1282' '2569_' '25_' '211' '793'
 '3484' '2072' '3050' '2162' '3402' '2753' '27_' '1718' '3260' '84' '2311'
 '1832' '3010' '733' '4241' '2461' '1749' '3200' '663_' '2185' '3009'
 '359' '2015' '1523' '1199' '1015' '1989' '281' '559' '3545' '779' '192'
 '-2_' '2323' '21_' '1471' '3529' '2697' '1014' '3179' '1332' '3112' '829'
 '4022' '4023' '531' '1511' '3092' '3191' '3621' '28_' '2300' '72' '398'
 '1473' '3043' '4216' '2903' '-1_' '4042' '1323_' '2438' '809' '1370'
 '1204' '2167' '4011' '2533' '1663' '1018' '3316' '2529' '2488' '1243'
 '845' '337' '290' '674' '2450' '3738' '1792' '2570' '775' '482' '1706'
 '2493' '3623' '3031' '2794_' '2219_' '758_' '1849' '1953' '2657' '4043'
 '2694' '519' '2677' '2609' '4326' '823' '1608' '2860' '4219' '1531'
 '4024' '1673' '1685' '2587' '3489' '749' '848_' '3845' '1060' '2573'
 '328' '2585' '2230' '3739' '2628' '996' '372' '3340' '3177' '1766' '1359'
 '1263' '3632' '2793' '3245' '2317' '2237_' '847' '3978' '2737' '1192'
 '1481' '271' '2286' '3944' '1478' '3749' '2959' '3097_' '3511' '2196'
 '1874' '1553' '3847' '1598' '2314' '2955' '1691' '1636' '3708' '320'
 '2945' '4159' '4397' '3776' '-3_' '2148' '853' '1178' '196' '468' '2044'
 '1541' '1211' '3162' '3142' '2766' '2728' '2671' '2705' '4251' '3119'
 '4185' '683' '4302' '3447' '1852' '2131' '1699' '210' '577' '2351' '867'
 '1371' '4053' '3869' '3660' '3629' '3208' '2142' '2521' '450' '583' '876'
 '2560' '2578' '813' '4360' '1154' '2544' '4172' '2924' '4270' '2768'
 '3909' '26_' '3171' '2569' '887' '4249' '2950' '107' '2506' '2810' '2873'
 '2262' '1890' '3078' '3865' '2777' '223' '2239' '2756' '1181' '3972'
 '2334' '3900' '2492' '2729' '3750' '1825' '309' '2431' '3099' '2080'
 '3722' '1976' '933' '3060' '4278' '3212' '46' '4231' '3790' '473' '1536'
 '3955' '2324' '2381' '1177' '371' '2896' '3880' '2991' '4319' '1061'
 '662' '4144' '2006' '3115' '3751' '1861' '4262' '2913' '3766' '2707'
 '1087' '3329' '1086' '1087_' '2457' '1614' '3274' '3488' '2854' '238'
 '3706' '4095' '1329' '3370' '283' '1392' '1743' '974' '4388' '4211'
 '4281' '3415' '3815' '441' '94' '3499' '969' '106' '1004' '2956' '85'
 '4113' '615' '2801' '1172' '2553' '3495' '1295_' '1879' '1735' '3584'
 '3827' '3754' '532' '2376' '3636' '3763' '778' '2621' '804' '754' '2418'
 '3926' '3861_' '162' '2834' '2354' '3355' '523' '1606' '1443' '1354'
 '2142_' '1422' '2278' '4106' '3155' '1841' '1216' '1473_' '2534' '4002'
 '2875' '4344' '3894' '1256' '676' '4178' '399' '1180' '86' '4037' '1150'
 '2591' '2149' '3721' '1775' '2260' '3707' '4292' '1820' '1480' '430'
 '3920_' '1389' '3336' '3392' '3688' '221' '2047']

 Changed_Credit_Limit ['11.27' '5.42' '7.42' ... '25.16' '22.08' '-3.41']

 Num_Credit_Inquiries [4.000e+00 2.000e+00 3.000e+00 5.000e+00 9.000e+00 8.0
00e+00 7.000e+00
```

6.000e+00 0.000e+00 1.000e+00 1.000e+01 1.050e+03 1.700e+01 1.936e+03
1.300e+01 5.680e+02 1.100e+01 1.618e+03 5.250e+02 1.200e+01 1.400e+01
1.600e+01 1.500e+01 2.850e+02 1.241e+03 1.839e+03 1.265e+03 1.085e+03
1.194e+03 1.197e+03 9.960e+02 1.515e+03 5.160e+02 8.290e+02 1.760e+03
2.539e+03 3.300e+01 6.020e+02 9.550e+02 7.940e+02 2.079e+03 9.770e+02
4.600e+02 3.040e+02 8.600e+01 1.347e+03 2.161e+03 4.710e+02 1.529e+03
2.187e+03 1.970e+02 6.100e+01 1.395e+03 1.114e+03 3.210e+02 2.572e+03
1.479e+03 2.050e+02 1.890e+03 2.081e+03 9.460e+02 1.393e+03 1.203e+03
1.015e+03 3.610e+02 1.498e+03 1.562e+03 3.400e+01 2.128e+03 1.425e+03
2.207e+03 8.240e+02 5.360e+02 2.056e+03 5.280e+02 1.247e+03 3.990e+02
3.930e+02 2.210e+02 1.189e+03 2.328e+03 2.548e+03 2.330e+02 2.870e+02
2.580e+03 2.245e+03 1.173e+03 2.326e+03 1.278e+03 1.999e+03 7.950e+02
2.308e+03 4.920e+02 6.270e+02 1.029e+03 1.390e+03 1.710e+02 1.481e+03
1.561e+03 2.568e+03 2.461e+03 3.360e+02 1.302e+03 2.357e+03 2.203e+03
1.128e+03 3.280e+02 9.090e+02 9.690e+02 1.319e+03 2.334e+03 2.211e+03
2.564e+03 1.630e+02 2.542e+03 1.419e+03 1.506e+03 6.930e+02 1.073e+03
3.160e+02 1.132e+03 2.279e+03 4.640e+02 1.804e+03 1.634e+03 8.310e+02
1.068e+03 9.870e+02 3.630e+02 1.866e+03 1.942e+03 1.864e+03 1.599e+03
3.710e+02 3.730e+02 1.845e+03 1.889e+03 2.150e+03 1.216e+03 1.990e+02
8.070e+02 1.370e+03 8.810e+02 2.199e+03 5.940e+02 7.260e+02 7.250e+02
1.232e+03 6.200e+02 4.730e+02 1.744e+03 2.402e+03 1.266e+03 5.410e+02
1.955e+03 9.920e+02 4.240e+02 1.880e+03 1.600e+02 1.517e+03 1.500e+03
1.607e+03 1.025e+03 1.123e+03 1.028e+03 9.750e+02 3.600e+02 1.330e+02
2.253e+03 6.600e+01 2.058e+03 1.033e+03 7.090e+02 1.768e+03 7.100e+01
2.190e+03 1.775e+03 1.150e+02 1.497e+03 2.241e+03 2.594e+03 7.830e+02
1.475e+03 8.760e+02 2.434e+03 1.726e+03 1.639e+03 2.444e+03 2.573e+03
9.670e+02 2.450e+02 2.313e+03 2.256e+03 5.300e+01 1.291e+03 9.300e+01
3.660e+02 7.470e+02 1.009e+03 4.430e+02 2.780e+02 1.375e+03 1.950e+03
2.362e+03 1.632e+03 2.720e+02 1.175e+03 1.058e+03 1.465e+03 1.337e+03
1.970e+03 1.795e+03 1.066e+03 2.324e+03 1.346e+03 5.810e+02 2.047e+03
1.370e+02 1.352e+03 1.204e+03 2.059e+03 6.280e+02 1.826e+03 8.160e+02
2.219e+03 6.300e+01 1.893e+03 1.500e+02 1.345e+03 3.370e+02 4.550e+02
8.530e+02 1.940e+03 2.323e+03 1.433e+03 6.370e+02 1.485e+03 1.740e+03
1.908e+03 1.380e+03 2.202e+03 1.310e+03 6.550e+02 1.299e+03 2.588e+03
1.549e+03 5.290e+02 9.170e+02 2.890e+02 9.190e+02 2.366e+03 2.310e+03
5.210e+02 1.057e+03 9.840e+02 1.037e+03 1.871e+03 2.090e+02 1.931e+03
9.910e+02 9.570e+02 1.707e+03 4.770e+02 1.340e+03 7.530e+02 1.525e+03
7.800e+01 2.515e+03 1.603e+03 1.992e+03 1.396e+03 1.833e+03 1.867e+03
1.906e+03 1.417e+03 2.022e+03 1.991e+03 2.191e+03 1.253e+03 8.370e+02
1.640e+02 1.129e+03 7.110e+02 1.588e+03 7.280e+02 1.160e+02 9.200e+02
5.420e+02 6.300e+02 1.309e+03 1.688e+03 5.990e+02 1.787e+03 1.870e+03
2.027e+03 2.039e+03 1.158e+03 1.221e+03 1.153e+03 2.302e+03 1.841e+03
3.090e+02 1.650e+03 8.030e+02 5.630e+02 1.389e+03 6.990e+02 1.690e+03
5.300e+02 2.520e+02 1.793e+03 1.930e+02 1.951e+03 1.721e+03 2.397e+03
6.210e+02 6.920e+02 2.446e+03 1.560e+03 1.362e+03 6.750e+02 2.069e+03
1.619e+03 1.402e+03 1.661e+03 1.230e+03 2.553e+03 1.244e+03 6.620e+02
1.521e+03 1.584e+03 2.275e+03 4.120e+02 2.433e+03 1.905e+03 2.494e+03
1.530e+03 8.500e+02 1.740e+02 1.053e+03 1.179e+03 1.535e+03 1.011e+03
1.202e+03 1.442e+03 5.140e+02 3.840e+02 1.220e+03 1.807e+03 2.019e+03
2.285e+03 1.447e+03 1.762e+03 9.140e+02 4.960e+02 1.796e+03 2.074e+03
5.660e+02 4.210e+02 1.290e+02 3.500e+02 1.366e+03 1.805e+03 1.662e+03
1.672e+03 2.460e+03 8.670e+02 3.800e+02 5.220e+02 8.870e+02 1.311e+03
2.155e+03 1.713e+03 1.693e+03 1.103e+03 1.139e+03 4.620e+02 2.470e+02
1.860e+02 2.107e+03 4.090e+02 8.080e+02 1.137e+03 6.790e+02 1.450e+03
2.600e+02 2.423e+03 2.260e+03 3.250e+02 5.200e+01 1.543e+03 4.390e+02
1.288e+03 1.980e+02 2.082e+03 8.110e+02 1.464e+03 1.042e+03 4.880e+02

```
 7.700e+02 6.230e+02 6.700e+01 2.286e+03 4.570e+02 1.072e+03 1.581e+03
 4.400e+02 2.457e+03 8.580e+02 2.331e+03 3.950e+02 2.246e+03 2.680e+02
 1.554e+03 2.169e+03 1.727e+03 1.487e+03 2.531e+03 3.490e+02 5.130e+02
 2.358e+03 1.269e+03 2.458e+03 1.816e+03 2.120e+03 1.329e+03 1.198e+03
 2.293e+03 4.680e+02 7.720e+02 1.598e+03 7.600e+01 2.184e+03 1.582e+03
 5.380e+02 1.547e+03 6.420e+02 2.589e+03 2.980e+02 1.387e+03 1.456e+03
 4.500e+02 5.760e+02 2.235e+03 2.017e+03 5.830e+02 1.007e+03 7.100e+02
 2.386e+03 2.294e+03 6.150e+02 1.850e+02 8.260e+02 2.292e+03 7.770e+02
 5.860e+02 2.547e+03 1.041e+03 2.317e+03 1.483e+03 1.172e+03 2.407e+03
 1.399e+03 1.480e+03 4.500e+01 3.180e+02 1.626e+03 1.654e+03 2.254e+03
 9.630e+02 9.790e+02 2.640e+02 1.167e+03 1.282e+03 1.941e+03 1.920e+03
 2.315e+03 4.260e+02 9.000e+02 1.188e+03 2.464e+03 2.363e+03 1.343e+03
 1.227e+03 2.990e+02 1.220e+02 2.185e+03 2.098e+03 1.531e+03 2.118e+03
 2.181e+03 1.868e+03 2.760e+02 8.460e+02 2.453e+03 5.400e+01 7.690e+02
 1.513e+03 1.579e+03 2.322e+03 7.550e+02 1.601e+03 1.540e+03 2.164e+03
 1.772e+03 1.133e+03 1.647e+03 2.537e+03 2.134e+03 1.263e+03 1.385e+03
 1.829e+03 6.850e+02 1.855e+03 7.900e+02 1.545e+03 1.573e+03 1.541e+03
 1.697e+03 1.168e+03 2.351e+03 1.000e+03 2.387e+03 1.089e+03 2.445e+03
 2.390e+03 2.091e+03 2.544e+03 1.275e+03 1.046e+03 4.840e+02 2.290e+03
 2.506e+03 1.086e+03 2.338e+03 8.770e+02 2.087e+03 6.430e+02 1.460e+03
 2.162e+03 1.335e+03 1.226e+03 2.221e+03 2.395e+03 1.640e+03 2.370e+03
 1.191e+03 2.061e+03 2.262e+03 1.117e+03 2.240e+03 1.303e+03 1.998e+03
 4.930e+02 1.429e+03 1.324e+03 6.030e+02 1.234e+03 7.430e+02 6.780e+02
 4.850e+02 1.155e+03 1.126e+03 1.333e+03 7.960e+02 9.760e+02 2.297e+03
 1.308e+03 1.633e+03 1.314e+03 9.950e+02 2.290e+02 3.850e+02 6.000e+01
 2.020e+02 1.682e+03 1.084e+03 3.150e+02 1.954e+03 2.381e+03 1.656e+03
 1.157e+03 1.472e+03 2.266e+03 2.469e+03 1.192e+03 3.030e+02 7.760e+02
 1.236e+03 1.836e+03 9.450e+02 1.761e+03 1.097e+03 1.988e+03 1.657e+03
 1.067e+03 2.120e+02 1.843e+03 1.017e+03 1.368e+03 2.264e+03 7.320e+02
 2.418e+03 1.594e+03 1.331e+03 2.660e+02 7.150e+02 2.228e+03 2.551e+03
 2.030e+02 6.970e+02 1.899e+03 1.580e+03 2.600e+01 3.570e+02 1.182e+03
 1.609e+03 1.788e+03 9.240e+02 8.400e+02 9.390e+02 2.498e+03 1.808e+03
 2.900e+01 2.374e+03 2.307e+03 2.178e+03 2.452e+03 2.144e+03 9.490e+02
 1.677e+03 1.605e+03 2.373e+03 2.492e+03 2.106e+03 1.786e+03 1.664e+03
 1.039e+03 5.150e+02 1.063e+03 5.600e+02 1.255e+03 8.910e+02 2.124e+03
 1.568e+03 1.815e+03 8.060e+02 6.580e+02 1.459e+03 2.356e+03 4.450e+02
 4.080e+02 1.246e+03 2.042e+03 2.570e+02 9.990e+02 2.165e+03 8.540e+02
 2.560e+02 3.540e+02 1.580e+02 1.527e+03 5.470e+02 1.782e+03 1.820e+03
 2.536e+03 6.340e+02 2.344e+03 1.136e+03 2.431e+03 2.340e+03 9.290e+02
 1.523e+03 1.274e+03 1.286e+03 1.315e+03 6.410e+02 1.702e+03 6.400e+01
 1.450e+02 2.426e+03 4.740e+02 1.318e+03 2.650e+02 2.518e+03 2.157e+03
 2.131e+03 2.301e+03 1.262e+03 9.150e+02 1.566e+03 7.980e+02 1.608e+03
 1.520e+02 3.980e+02 1.252e+03 1.548e+03 1.622e+03 1.410e+03 1.834e+03
 7.920e+02 9.020e+02 5.970e+02 1.989e+03 2.514e+03 7.360e+02 7.190e+02
 2.021e+03 1.019e+03 6.090e+02 3.440e+02 2.170e+03 1.938e+03 1.440e+03
 1.659e+03 2.405e+03 1.001e+03 6.320e+02 1.297e+03 1.054e+03 2.367e+03
 1.963e+03 2.583e+03 7.840e+02 1.876e+03 2.244e+03 1.377e+03 2.239e+03
 1.862e+03 2.001e+03 4.690e+02 2.630e+02 1.233e+03 2.505e+03 7.580e+02
 1.388e+03 1.854e+03 6.080e+02 1.187e+03 1.180e+02 1.918e+03 1.801e+03
 1.888e+03 7.620e+02 2.072e+03 1.361e+03 3.100e+02]

 Credit_Mix ['_' 'Good' 'Standard' 'Bad']

 Outstanding_Debt ['809.98' '605.03' '1303.01' ... '3571.7_' '3571.7' '502.3
8']
```

```
 Credit_Utilization_Ratio [26.82261962 22.53759303 24.46403064 ... 40.565630
96 41.25552226
 34.19246265]

 Credit_History_Age ['22 Years and 1 Months' '22 Years and 7 Months' '26 Yea
rs and 7 Months'
 '26 Years and 8 Months' '26 Years and 9 Months' '26 Years and 11 Months'
 '27 Years and 0 Months' '27 Years and 2 Months' '17 Years and 9 Months'
 '18 Years and 1 Months' '18 Years and 2 Months' '18 Years and 4 Months'
 '17 Years and 3 Months' '17 Years and 4 Months' '17 Years and 5 Months'
 '17 Years and 6 Months' '17 Years and 7 Months' '17 Years and 8 Months'
 '17 Years and 10 Months' '30 Years and 7 Months' '30 Years and 8 Months'
 '30 Years and 9 Months' '30 Years and 10 Months' '30 Years and 11 Months'
 '31 Years and 2 Months' '14 Years and 8 Months' '14 Years and 10 Months'
 '15 Years and 2 Months' '21 Years and 5 Months' '21 Years and 8 Months'
 '21 Years and 9 Months' '21 Years and 11 Months' '26 Years and 6 Months'
 '19 Years and 3 Months' '19 Years and 5 Months' '19 Years and 6 Months'
 '19 Years and 7 Months' '19 Years and 8 Months' '25 Years and 5 Months'
 '25 Years and 6 Months' '25 Years and 7 Months' '25 Years and 9 Months'
 '25 Years and 10 Months' '26 Years and 10 Months' '27 Years and 1 Months'
 '27 Years and 3 Months' '27 Years and 4 Months' '27 Years and 5 Months'
 '9 Years and 0 Months' '9 Years and 2 Months' '9 Years and 3 Months'
 '18 Years and 6 Months' '18 Years and 8 Months' '18 Years and 9 Months'
 '16 Years and 10 Months' '16 Years and 11 Months' '17 Years and 0 Months'
 '17 Years and 1 Months' '17 Years and 2 Months' '6 Years and 5 Months'
 '6 Years and 6 Months' '6 Years and 7 Months' '6 Years and 8 Months'
 '6 Years and 10 Months' '7 Years and 0 Months' '18 Years and 3 Months'
 '18 Years and 5 Months' '18 Years and 7 Months' '10 Years and 2 Months'
 '10 Years and 3 Months' '10 Years and 4 Months' '10 Years and 6 Months'
 '10 Years and 7 Months' '10 Years and 8 Months' '12 Years and 4 Months'
 '12 Years and 7 Months' '12 Years and 8 Months' '12 Years and 6 Months'
 '13 Years and 8 Months' '14 Years and 0 Months' '14 Years and 1 Months'
 '14 Years and 2 Months' '19 Years and 4 Months' '30 Years and 3 Months'
 '30 Years and 4 Months' '30 Years and 5 Months' '8 Years and 9 Months'
 '8 Years and 10 Months' '8 Years and 11 Months' '19 Years and 0 Months'
 '19 Years and 2 Months' '8 Years and 8 Months' '13 Years and 1 Months'
 '13 Years and 2 Months' '13 Years and 7 Months' '21 Years and 6 Months'
 '21 Years and 10 Months' '25 Years and 8 Months' '25 Years and 11 Months'
 '26 Years and 0 Months' '13 Years and 4 Months' '13 Years and 5 Months'
 '13 Years and 6 Months' '13 Years and 9 Months' '27 Years and 11 Months'
 '28 Years and 1 Months' '28 Years and 3 Months' '28 Years and 4 Months'
 '28 Years and 5 Months' '28 Years and 6 Months' '7 Years and 10 Months'
 '7 Years and 11 Months' '8 Years and 0 Months' '8 Years and 2 Months'
 '8 Years and 3 Months' '8 Years and 4 Months' '24 Years and 3 Months'
 '24 Years and 7 Months' '24 Years and 8 Months' '1 Years and 5 Months'
 '1 Years and 6 Months' '1 Years and 7 Months' '1 Years and 8 Months'
 '11 Years and 3 Months' '11 Years and 4 Months' '11 Years and 5 Months'
 '31 Years and 0 Months' '31 Years and 1 Months' '10 Years and 9 Months'
 '14 Years and 3 Months' '14 Years and 4 Months' '14 Years and 5 Months'
 '14 Years and 6 Months' '20 Years and 9 Months' '21 Years and 0 Months'
 '21 Years and 1 Months' '21 Years and 2 Months' '21 Years and 3 Months'
 '0 Years and 5 Months' '0 Years and 6 Months' '0 Years and 9 Months'
 '31 Years and 8 Months' '31 Years and 9 Months' '31 Years and 11 Months'
 '32 Years and 1 Months' '12 Years and 5 Months' '12 Years and 9 Months'
 '12 Years and 10 Months' '12 Years and 11 Months' '13 Years and 0 Months'
 '11 Years and 6 Months' '11 Years and 9 Months' '24 Years and 9 Months'
```

'24 Years and 10 Months' '24 Years and 11 Months' '25 Years and 1 Months'
'25 Years and 2 Months' '25 Years and 3 Months' '10 Years and 1 Months'
'19 Years and 1 Months' '31 Years and 3 Months' '31 Years and 5 Months'
'31 Years and 6 Months' '31 Years and 7 Months' '13 Years and 3 Months'
'5 Years and 2 Months' '5 Years and 3 Months' '5 Years and 4 Months'
'5 Years and 5 Months' '5 Years and 6 Months' '5 Years and 8 Months'
'2 Years and 11 Months' '3 Years and 1 Months' '3 Years and 2 Months'
'3 Years and 3 Months' '3 Years and 5 Months' '3 Years and 6 Months'
'24 Years and 6 Months' '16 Years and 4 Months' '16 Years and 5 Months'
'16 Years and 7 Months' '16 Years and 8 Months' '16 Years and 9 Months'
'22 Years and 11 Months' '23 Years and 3 Months' '23 Years and 4 Months'
'23 Years and 5 Months' '8 Years and 5 Months' '8 Years and 7 Months'
'31 Years and 4 Months' '4 Years and 6 Months' '4 Years and 9 Months'
'4 Years and 11 Months' '5 Years and 0 Months' '32 Years and 8 Months'
'32 Years and 9 Months' '32 Years and 10 Months' '32 Years and 11 Months'
'33 Years and 0 Months' '33 Years and 2 Months' '33 Years and 3 Months'
'12 Years and 2 Months' '12 Years and 3 Months' '29 Years and 9 Months'
'29 Years and 11 Months' '30 Years and 0 Months' '30 Years and 2 Months'
'26 Years and 1 Months' '26 Years and 2 Months' '33 Years and 4 Months'
'26 Years and 3 Months' '17 Years and 11 Months' '18 Years and 0 Months'
'7 Years and 6 Months' '7 Years and 7 Months' '7 Years and 9 Months'
'11 Years and 2 Months' '11 Years and 8 Months' '28 Years and 8 Months'
'28 Years and 9 Months' '28 Years and 10 Months' '29 Years and 2 Months'
'29 Years and 3 Months' '29 Years and 4 Months' '29 Years and 5 Months'
'29 Years and 6 Months' '29 Years and 8 Months' '13 Years and 10 Months'
'1 Years and 4 Months' '1 Years and 11 Months' '33 Years and 1 Months'
'33 Years and 6 Months' '33 Years and 7 Months' '33 Years and 8 Months'
'26 Years and 4 Months' '31 Years and 10 Months' '32 Years and 0 Months'
'32 Years and 2 Months' '24 Years and 4 Months' '24 Years and 5 Months'
'5 Years and 9 Months' '5 Years and 10 Months' '6 Years and 0 Months'
'6 Years and 1 Months' '6 Years and 3 Months' '19 Years and 9 Months'
'16 Years and 6 Months' '21 Years and 7 Months' '22 Years and 0 Months'
'22 Years and 2 Months' '20 Years and 2 Months' '20 Years and 3 Months'
'20 Years and 6 Months' '20 Years and 7 Months' '20 Years and 8 Months'
'15 Years and 4 Months' '15 Years and 6 Months' '15 Years and 8 Months'
'15 Years and 10 Months' '15 Years and 11 Months' '2 Years and 3 Months'
'2 Years and 4 Months' '2 Years and 5 Months' '2 Years and 7 Months'
'2 Years and 10 Months' '4 Years and 10 Months' '5 Years and 1 Months'
'1 Years and 9 Months' '1 Years and 10 Months' '2 Years and 0 Months'
'15 Years and 5 Months' '15 Years and 7 Months' '16 Years and 3 Months'
'9 Years and 4 Months' '9 Years and 5 Months' '9 Years and 8 Months'
'9 Years and 9 Months' '27 Years and 7 Months' '27 Years and 8 Months'
'27 Years and 9 Months' '27 Years and 10 Months' '28 Years and 0 Months'
'28 Years and 2 Months' '18 Years and 11 Months' '8 Years and 1 Months'
'5 Years and 7 Months' '6 Years and 2 Months' '12 Years and 0 Months'
'26 Years and 5 Months' '29 Years and 0 Months' '11 Years and 7 Months'
'11 Years and 10 Months' '11 Years and 11 Months' '21 Years and 4 Months'
'22 Years and 9 Months' '22 Years and 10 Months' '23 Years and 0 Months'
'23 Years and 1 Months' '23 Years and 2 Months' '16 Years and 0 Months'
'16 Years and 1 Months' '16 Years and 2 Months' '14 Years and 9 Months'
'14 Years and 11 Months' '15 Years and 0 Months' '20 Years and 4 Months'
'20 Years and 5 Months' '22 Years and 4 Months' '22 Years and 5 Months'
'22 Years and 6 Months' '22 Years and 8 Months' '18 Years and 10 Months'
'8 Years and 6 Months' '9 Years and 1 Months' '5 Years and 11 Months'
'25 Years and 0 Months' '25 Years and 4 Months' '15 Years and 3 Months'
'15 Years and 9 Months' '32 Years and 3 Months' '32 Years and 5 Months'

```
 '32 Years and 6 Months' '33 Years and 5 Months' '6 Years and 11 Months'
 '7 Years and 1 Months' '7 Years and 2 Months' '7 Years and 4 Months'
 '7 Years and 5 Months' '23 Years and 7 Months' '23 Years and 9 Months'
 '10 Years and 10 Months' '15 Years and 1 Months' '20 Years and 11 Months'
 '30 Years and 6 Months' '7 Years and 3 Months' '22 Years and 3 Months'
 '9 Years and 10 Months' '10 Years and 0 Months' '28 Years and 7 Months'
 '28 Years and 11 Months' '2 Years and 2 Months' '2 Years and 6 Months'
 '2 Years and 9 Months' '23 Years and 8 Months' '23 Years and 10 Months'
 '23 Years and 11 Months' '9 Years and 7 Months' '29 Years and 7 Months'
 '9 Years and 6 Months' '10 Years and 11 Months' '11 Years and 0 Months'
 '11 Years and 1 Months' '7 Years and 8 Months' '24 Years and 1 Months'
 '24 Years and 2 Months' '19 Years and 11 Months' '20 Years and 0 Months'
 '20 Years and 1 Months' '29 Years and 1 Months' '27 Years and 6 Months'
 '6 Years and 4 Months' '0 Years and 2 Months' '0 Years and 4 Months'
 '0 Years and 7 Months' '0 Years and 8 Months' '9 Years and 11 Months'
 '13 Years and 11 Months' '12 Years and 1 Months' '29 Years and 10 Months'
 '10 Years and 5 Months' '30 Years and 1 Months' '0 Years and 1 Months'
 '3 Years and 4 Months' '3 Years and 8 Months' '4 Years and 7 Months'
 '4 Years and 8 Months' '32 Years and 7 Months' '3 Years and 7 Months'
 '3 Years and 9 Months' '3 Years and 10 Months' '6 Years and 9 Months'
 '19 Years and 10 Months' '14 Years and 7 Months' '1 Years and 0 Months'
 '1 Years and 1 Months' '1 Years and 2 Months' '4 Years and 4 Months'
 '4 Years and 5 Months' '3 Years and 11 Months' '4 Years and 1 Months'
 '4 Years and 3 Months' '1 Years and 3 Months' '2 Years and 1 Months'
 '4 Years and 2 Months' '32 Years and 4 Months' '23 Years and 6 Months'
 '24 Years and 0 Months' '0 Years and 10 Months' '4 Years and 0 Months'
 '20 Years and 10 Months' '0 Years and 11 Months' '2 Years and 8 Months'
 '3 Years and 0 Months' '0 Years and 3 Months']

 Payment_of_Min_Amount ['No' 'NM' 'Yes']

 Total_EMI_per_month [   49.57494921    18.81621457   246.99231945 ...    6
0.96477237
 12112.            35.10402261]

 Amount_invested_monthly ['80.41529543900253' '178.3440674122349' '104.29182
5168246' ...
 '54.18595028760385' '24.02847744864441' '167.1638651610451']

 Payment_Behaviour ['High_spent_Small_value_payments' 'Low_spent_Small_value
_payments'
 'High_spent_Large_value_payments' '!@9#%8'
 'Low_spent_Large_value_payments' 'High_spent_Medium_value_payments'
 'Low_spent_Medium_value_payments']

 Monthly_Balance ['312.49408867943663' '244.5653167062043' '470.690626925291
84' ...
 '496.651610435322' '516.8090832742814' '393.6736955618808']

 Credit_Score ['Good' 'Standard' 'Poor']
```

In [10]:

```python
def preprocess_data(data, mvi_groupby=None, mvi_customval=None, column=None,
                    unwanted_value_replace=None, unwanted_value_strip=None,
    # Stripping unwanted values that might be at the beginning or end of the
    if unwanted_value_strip is not None:
        if data[column].dtype == object:
```

```
            data[column] = data[column].str.strip(unwanted_value_strip)
            print(f"\nTrailing & leading '{unwanted_value_strip}' are remove

    # Replacing unwanted value with NaN
    if unwanted_value_replace is not None:
        data[column] = data[column].replace(unwanted_value_replace, np.nan)
        print(f"\nUnwanted value '{unwanted_value_replace}' is replaced with

    # Performing missing value imputation (MVI) using mode after grouping da
    if mvi_groupby and column:
        data[column] = data[column].replace('', np.nan)
        group_mode = data.groupby(mvi_groupby)[column].transform(lambda x: x
        data[column] = data[column].fillna(group_mode)
        print("\nMissing values imputed with group mode")

    # Performing missing value imputation using a user-provided custom value
    if mvi_customval is not None:
        data[column] = data[column].replace('', np.nan)
        data[column].replace([np.NaN], mvi_customval, inplace=True)
        print(f"\nMissing values are replaced with '{mvi_customval}'")

    # Changing the data type of the column based on user-provided data type
    if datatype is not None:
        data[column] = data[column].astype(datatype)
        print(f"\nDatatype of '{column}' is changed to {datatype}")

    print('-----------------------------------------------------')


# Example Usage
print("Column: Name")
preprocess_data(data=df, column='Name', mvi_groupby='Customer_ID')

print("Column: Type_of_Loan")
preprocess_data(data=df, column='Type_of_Loan', mvi_customval='Not Specified
```

```
Column: Name

Missing values imputed with group mode
-----------------------------------------------------
Column: Type_of_Loan

Missing values are replaced with 'Not Specified'
-----------------------------------------------------
```

In [11]:

```
print("Column: SSN")
preprocess_data(data = df,
column = 'SSN',
unwanted_value_replace = '#F%$D@*&8',
mvi_groupby = 'Customer_ID')
print("Column: Occupation")
preprocess_data(data = df,
column = 'Occupation',
unwanted_value_replace = '_______',
mvi_groupby = 'Customer_ID',)
```

```
print("Column: Credit_Mix")
preprocess_data(data = df,
column = 'Credit_Mix',
unwanted_value_replace = '_',
mvi_groupby = 'Customer_ID')

print("Column: Payment_Behaviour")
preprocess_data(data = df,
column = 'Payment_Behaviour',
unwanted_value_replace = '!@9#%8',
mvi_groupby = 'Customer_ID')
```

```
Column: SSN

Unwanted value '#F%$D@*&8' is replaced with NaN

Missing values imputed with group mode
-----------------------------------------------------
Column: Occupation

Unwanted value '_______' is replaced with NaN

Missing values imputed with group mode
-----------------------------------------------------
Column: Credit_Mix

Unwanted value '_' is replaced with NaN

Missing values imputed with group mode
-----------------------------------------------------
Column: Payment_Behaviour

Unwanted value '!@9#%8' is replaced with NaN

Missing values imputed with group mode
-----------------------------------------------------
```

In [12]:
```
print("Column: Monthly_Inhand_Salary")
preprocess_data(data = df,
column = 'Monthly_Inhand_Salary',
mvi_groupby = 'Customer_ID')

print("Column: Num_Credit_Inquiries")
preprocess_data(data = df,
column = 'Num_Credit_Inquiries',
mvi_groupby = 'Customer_ID')
```

```
Column: Monthly_Inhand_Salary

Missing values imputed with group mode
-----------------------------------------------------
Column: Num_Credit_Inquiries

Missing values imputed with group mode
-----------------------------------------------------
```

In [13]:

```
print("Column: Age")
preprocess_data(data = df,
column = 'Age',
unwanted_value_strip = '_',
mvi_groupby = 'Customer_ID',
datatype = 'int')

print("Column: Annual_Income")
preprocess_data(data = df,
column = 'Annual_Income',
unwanted_value_strip = '_',
mvi_groupby = 'Customer_ID',
datatype = 'float')

print("Column: Outstanding_Debt")
preprocess_data(data = df,
column = 'Outstanding_Debt',
unwanted_value_strip = '_',
mvi_groupby = 'Customer_ID',
datatype = 'float')

print("Column: Amount_invested_monthly")
preprocess_data(data = df,
column = 'Amount_invested_monthly',
unwanted_value_strip = '_',
mvi_groupby = 'Customer_ID',
datatype = 'float')

print("Column: Num_of_Loan")
preprocess_data(data = df,
column = 'Num_of_Loan',
unwanted_value_strip = '_',
mvi_groupby = 'Customer_ID',
datatype = 'int')

print("Column: Num_of_Delayed_Payment")
preprocess_data(data = df,
column = 'Num_of_Delayed_Payment',
unwanted_value_strip = '_',
mvi_groupby = 'Customer_ID',
datatype = 'float')
```

```
Column: Age

Trailing & leading '_' are removed

Missing values imputed with group mode

Datatype of 'Age' is changed to int
--------------------------------------------------
Column: Annual_Income

Trailing & leading '_' are removed

Missing values imputed with group mode

Datatype of 'Annual_Income' is changed to float
--------------------------------------------------
Column: Outstanding_Debt

Trailing & leading '_' are removed

Missing values imputed with group mode

Datatype of 'Outstanding_Debt' is changed to float
--------------------------------------------------
Column: Amount_invested_monthly

Trailing & leading '_' are removed

Missing values imputed with group mode

Datatype of 'Amount_invested_monthly' is changed to float
--------------------------------------------------
Column: Num_of_Loan

Trailing & leading '_' are removed

Missing values imputed with group mode

Datatype of 'Num_of_Loan' is changed to int
--------------------------------------------------
Column: Num_of_Delayed_Payment

Trailing & leading '_' are removed

Missing values imputed with group mode

Datatype of 'Num_of_Delayed_Payment' is changed to float
--------------------------------------------------
```

In [14]:

```
print("Column: Changed_Credit_Limit")
preprocess_data(data = df,
column = 'Changed_Credit_Limit',
unwanted_value_strip = '_',
unwanted_value_replace = '_',
mvi_groupby = 'Customer_ID',
datatype = 'float')
```

```
print("Column: Monthly_Balance")
preprocess_data(data = df,
column = 'Monthly_Balance',
unwanted_value_strip = '_',
unwanted_value_replace = '__-333333333333333333333333333__',
mvi_groupby = 'Customer_ID',
datatype = 'float')
```

```
Column: Changed_Credit_Limit

Trailing & leading '_' are removed

Unwanted value '_' is replaced with NaN

Missing values imputed with group mode

Datatype of 'Changed_Credit_Limit' is changed to float
----------------------------------------------------
Column: Monthly_Balance

Trailing & leading '_' are removed

Unwanted value '__-333333333333333333333333333__' is replaced with NaN

Missing values imputed with group mode

Datatype of 'Monthly_Balance' is changed to float
----------------------------------------------------
```

In [15]:
```
# creating a function that picks the year and month and then combines them t
def credit_history_in_months(val):
    if pd.notnull(val):
        years = int(val.split(' ')[0])
        month = int(val.split(' ')[3])
        return (years*12)+month
    else:
        return val

# applying the function to the column
df['Credit_History_Age'] = df['Credit_History_Age'].apply(lambda x: credit_h

print('Column: Credit_History_Age')
preprocess_data(data = df,
column = 'Credit_History_Age',
mvi_groupby = 'Customer_ID')
```

```
Column: Credit_History_Age

Missing values imputed with group mode
----------------------------------------------------
```

In [16]:
```
df.isna().sum()
```

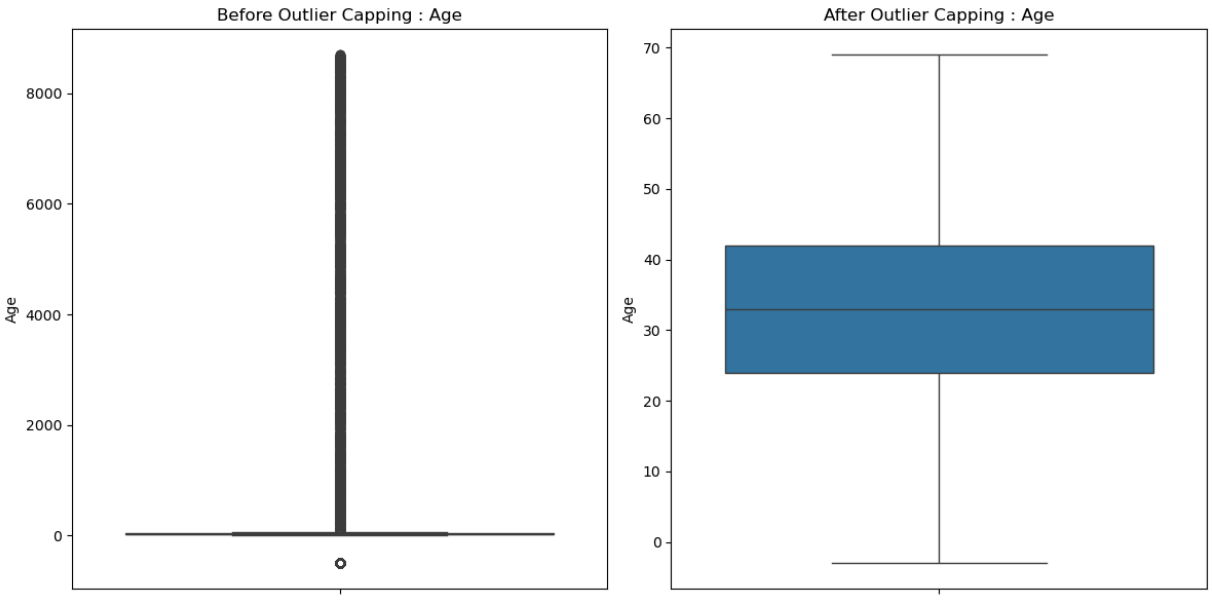
Out[16]:
```
ID                          0
Customer_ID                 0
Month                       0
Name                        0
Age                         0
SSN                         0
Occupation                  0
Annual_Income               0
Monthly_Inhand_Salary       0
Num_Bank_Accounts           0
Num_Credit_Card             0
Interest_Rate               0
Num_of_Loan                 0
Type_of_Loan                0
Delay_from_due_date         0
Num_of_Delayed_Payment      0
Changed_Credit_Limit        0
Num_Credit_Inquiries        0
Credit_Mix                  0
Outstanding_Debt            0
Credit_Utilization_Ratio    0
Credit_History_Age          0
Payment_of_Min_Amount       0
Total_EMI_per_month         0
Amount_invested_monthly     0
Payment_Behaviour           0
Monthly_Balance             0
Credit_Score                0
dtype: int64
```

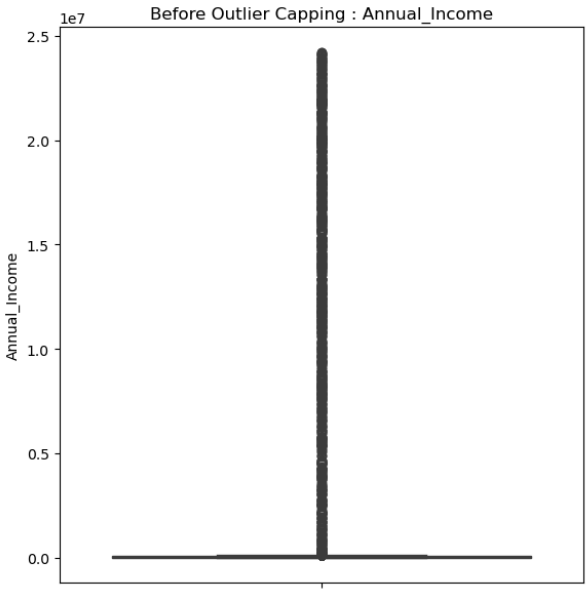
In [17]:
```
df.dtypes
```

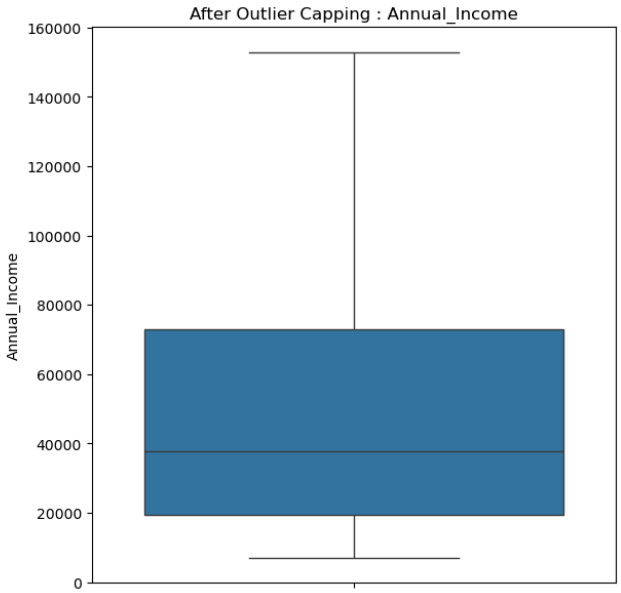
Out[17]:
```
ID                          object
Customer_ID                 object
Month                       object
Name                        object
Age                          int32
SSN                         object
Occupation                  object
Annual_Income              float64
Monthly_Inhand_Salary      float64
Num_Bank_Accounts            int64
Num_Credit_Card              int64
Interest_Rate                int64
Num_of_Loan                  int32
Type_of_Loan                object
Delay_from_due_date          int64
Num_of_Delayed_Payment     float64
Changed_Credit_Limit       float64
Num_Credit_Inquiries       float64
Credit_Mix                  object
Outstanding_Debt           float64
Credit_Utilization_Ratio   float64
Credit_History_Age         float64
Payment_of_Min_Amount       object
Total_EMI_per_month        float64
Amount_invested_monthly    float64
Payment_Behaviour           object
Monthly_Balance            float64
Credit_Score                object
dtype: object
```

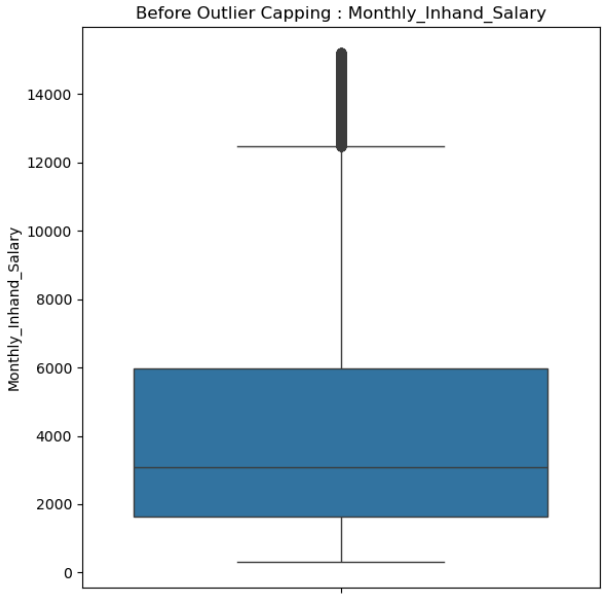
In [18]:
```python
def outlier_capping(data, threshold=1.5):
    # Making a copy of the input DataFrame
    data_copy = data.copy()

    # Creating an empty list to save the outlier indices
    outlier_indices = []

    # Calculating quartile 1 and 3 for every numerical column in the data
    for column in data_copy.columns:
        if pd.api.types.is_numeric_dtype(data_copy[column]):
            # Calculating quartiles
            Q1 = data_copy[column].quantile(0.25)
            Q3 = data_copy[column].quantile(0.75)

            # Calculating inter-quartile range
            IQR = Q3 - Q1

            # Defining the upper and lower outlier bounds
            lower_bound = Q1 - threshold * IQR
            upper_bound = Q3 + threshold * IQR

            # Identifying outliers
            outliers = data_copy[(data_copy[column] < lower_bound) | (data_c
            outlier_indices.extend(outliers.index)

            # Capping outliers
```

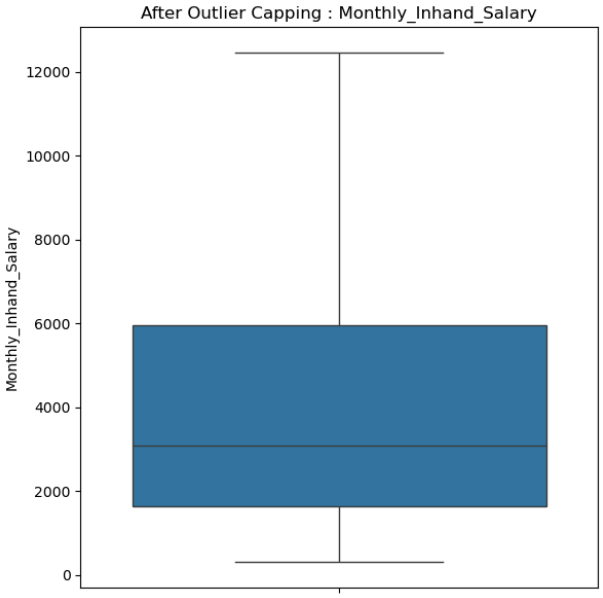
```
            data_copy[column] = np.where(data_copy[column] < lower_bound, lo
            data_copy[column] = np.where(data_copy[column] > upper_bound, up

    # Removing duplicates from the outlier_indices list
    outlier_indices = list(set(outlier_indices))

    # Returning the DataFrame with capped outliers
    return data_copy, outlier_indices

# Running the function to cap all the outliers in the data
df_clean, outliers = outlier_capping(df)
def outlier_capping_comparison(data, data_processed, outlier_indices):
    for column in data.columns:
        if pd.api.types.is_numeric_dtype(data[column]):
            plt.figure(figsize=(12, 6))

            # Plotting the numerical columns from the original DataFrame
            plt.subplot(1, 2, 1)
            sns.boxplot(data=data[column])
            plt.title(f'Before Outlier Capping : {column}')

            # Plot distribution of the numerical columns where outliers are
            plt.subplot(1, 2, 2)
            sns.boxplot(data=data_processed[column])
            plt.title(f'After Outlier Capping : {column}')

            # Showing output
            plt.tight_layout()
            plt.show()

# Running the function to compare the numerical columns before and after out
outlier_capping_comparison(data=df, data_processed=df_clean, outlier_indices
```

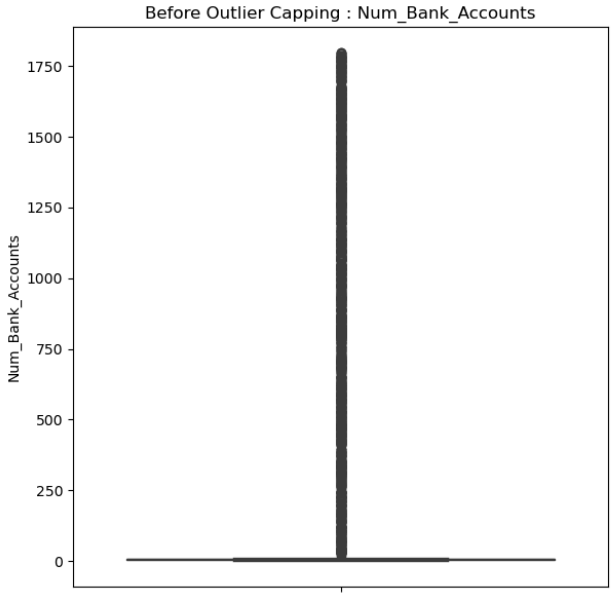


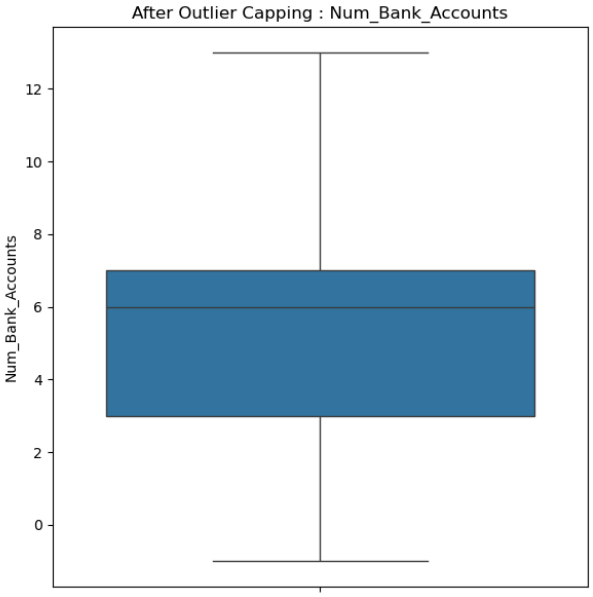

Before Outlier Capping : Annual_Income
1e7
2.5
2.0
1.5
1.0
0.5
0.0
Annual_Income

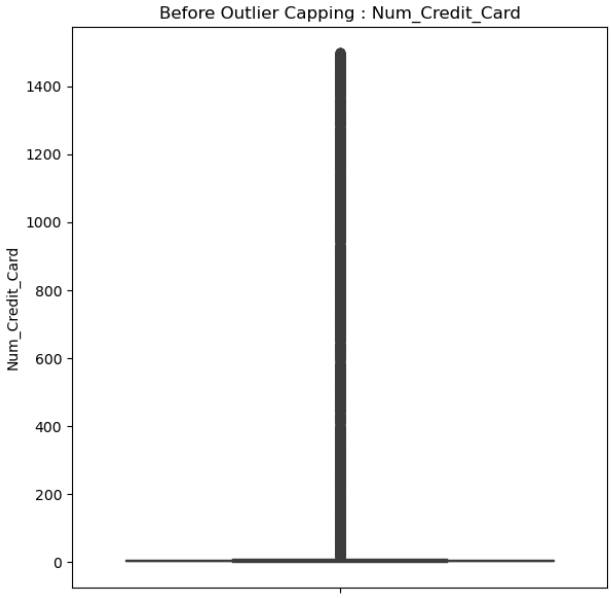

After Outlier Capping : Annual_Income
160000
140000
120000
100000
80000
60000
40000
20000
0
Annual_Income

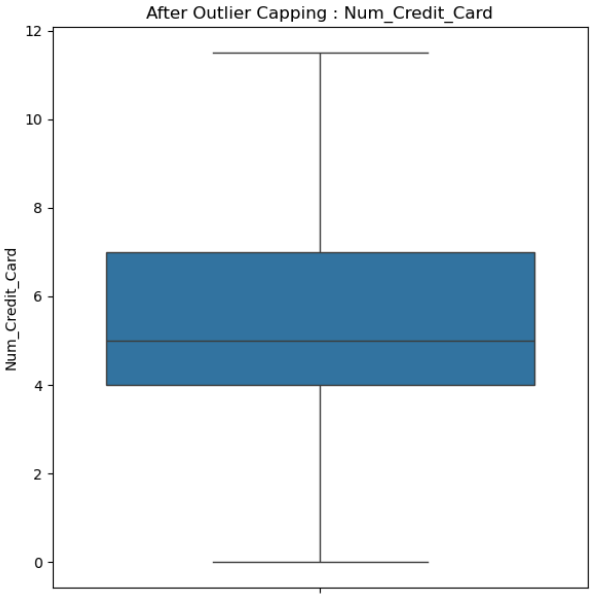

Before Outlier Capping : Monthly_Inhand_Salary
14000
12000
10000
8000
6000
4000
2000
0
Monthly_Inhand_Salary

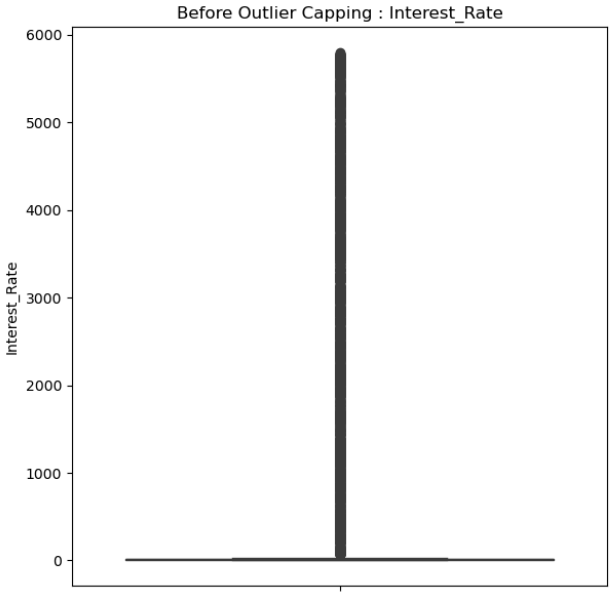

After Outlier Capping : Monthly_Inhand_Salary
12000
10000
8000
6000
4000
2000
0
Monthly_Inhand_Salary

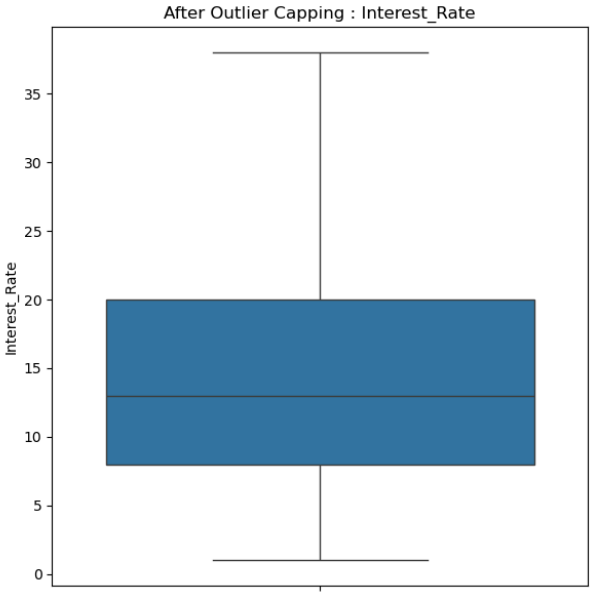

Before Outlier Capping : Num_Bank_Accounts
1750
1500
1250
1000
750
500
250
0
Num_Bank_Accounts

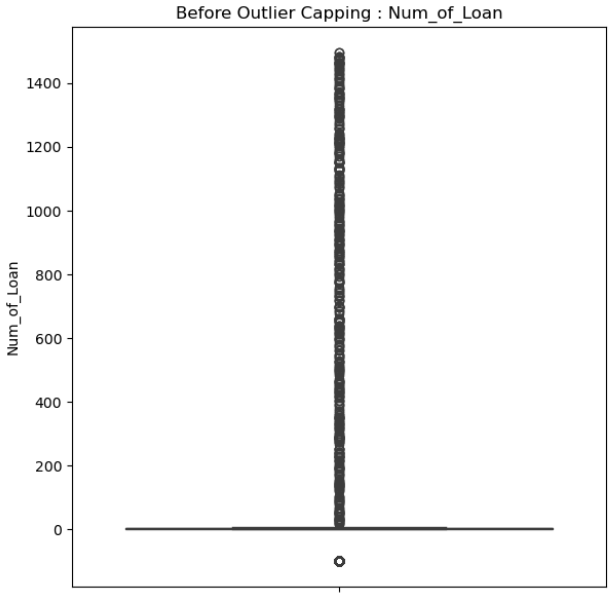

After Outlier Capping : Num_Bank_Accounts
12
10
8
6
4
2
0
Num_Bank_Accounts

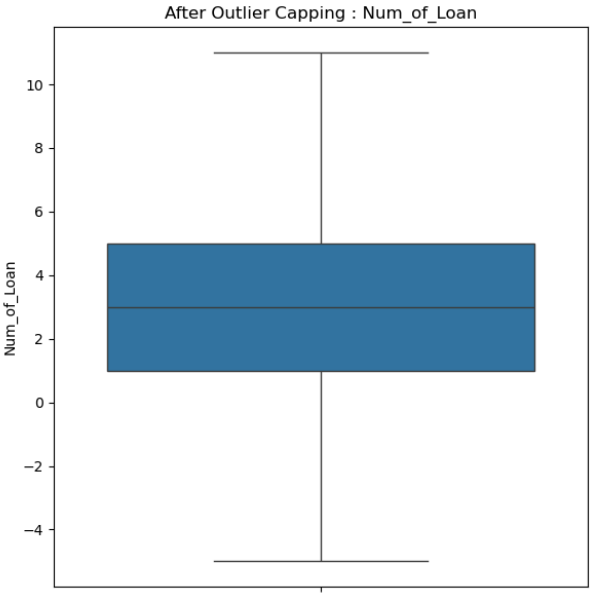

Before Outlier Capping : Num_Credit_Card
Num_Credit_Card
0
200
400
600
800
1000
1200
1400

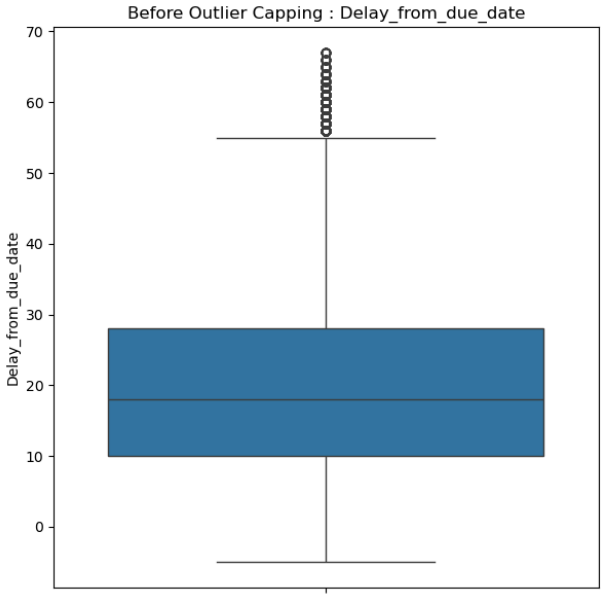

After Outlier Capping : Num_Credit_Card
Num_Credit_Card
0
2
4
6
8
10
12

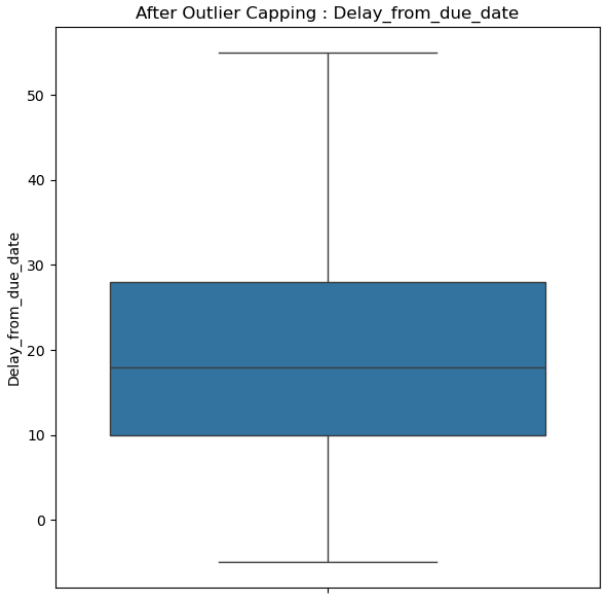

Before Outlier Capping : Interest_Rate
Interest_Rate
0
1000
2000
3000
4000
5000
6000

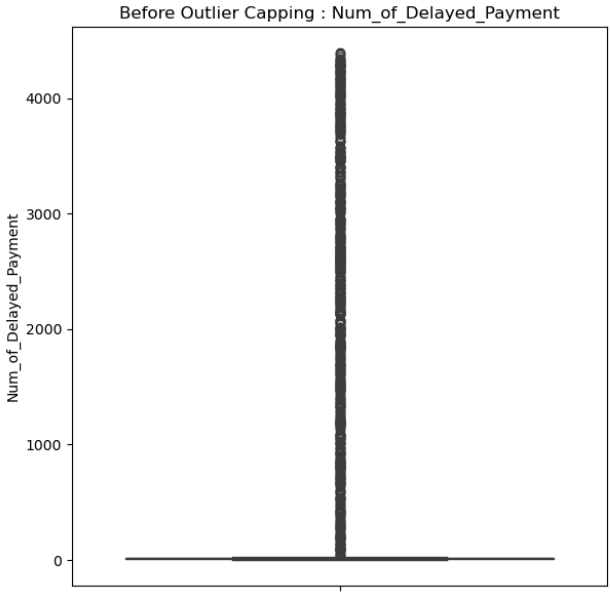

After Outlier Capping : Interest_Rate
Interest_Rate
0
5
10
15
20
25
30
35

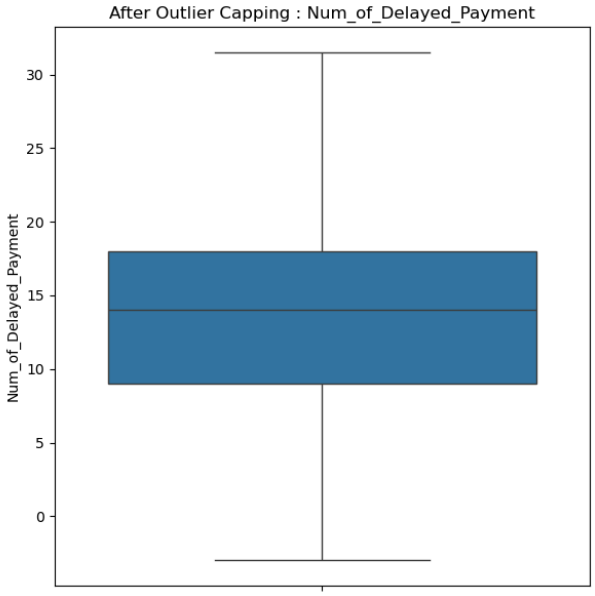

Before Outlier Capping : Num_of_Loan
Num_of_Loan
0
200
400
600
800
1000
1200
1400

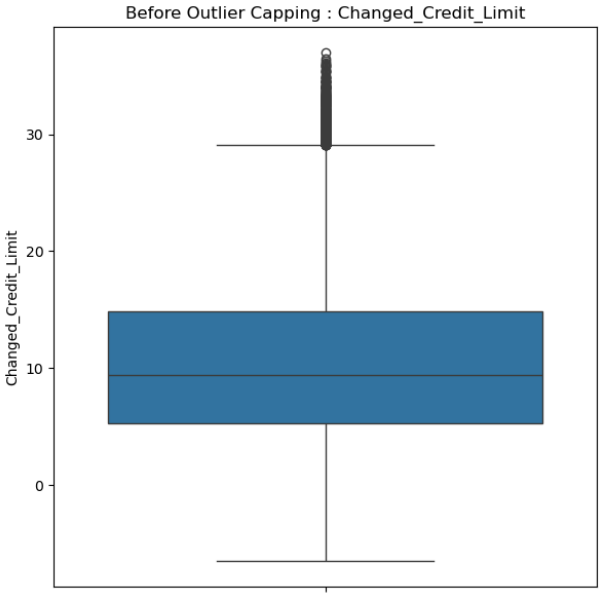

After Outlier Capping : Num_of_Loan
Num_of_Loan
−4
−2
0
2
4
6
8
10

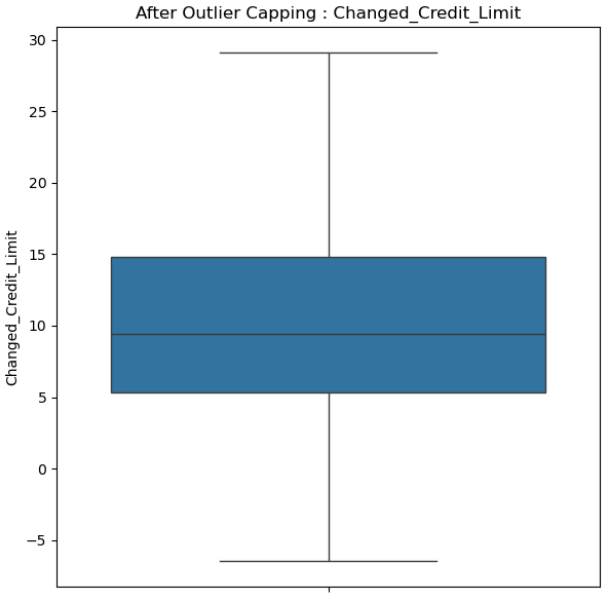

Before Outlier Capping : Delay_from_due_date
Delay_from_due_date
70
60
50
40
30
20
10
0

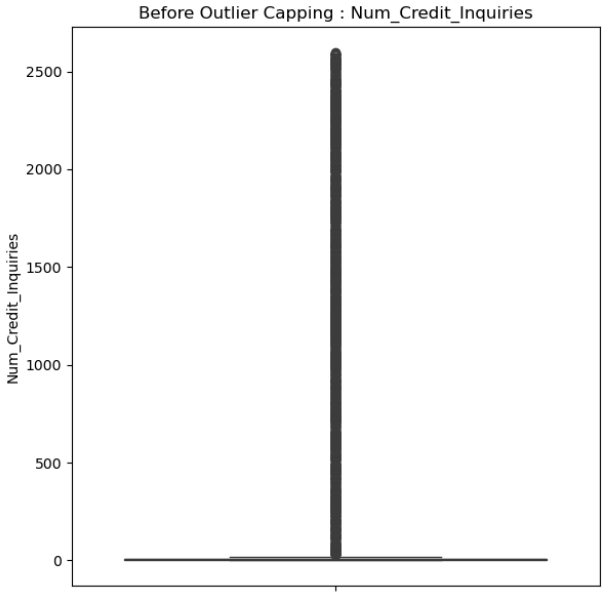

After Outlier Capping : Delay_from_due_date
Delay_from_due_date
50
40
30
20
10
0

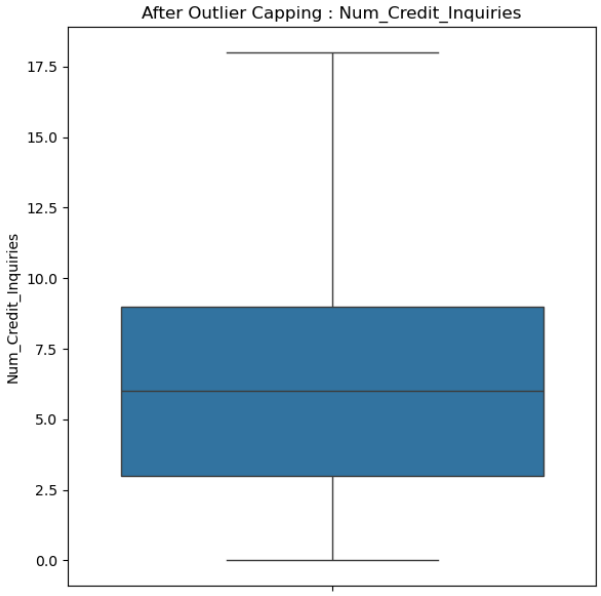

Before Outlier Capping : Num_of_Delayed_Payment
Num_of_Delayed_Payment
4000
3000
2000
1000
0

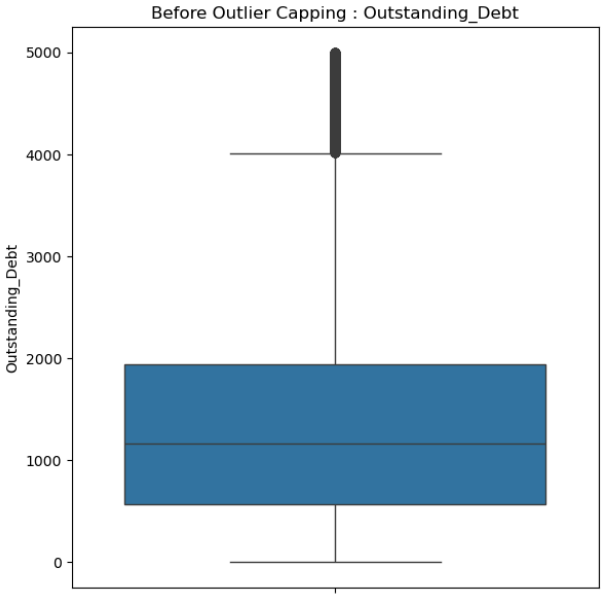

After Outlier Capping : Num_of_Delayed_Payment
Num_of_Delayed_Payment
30
25
20
15
10
5
0

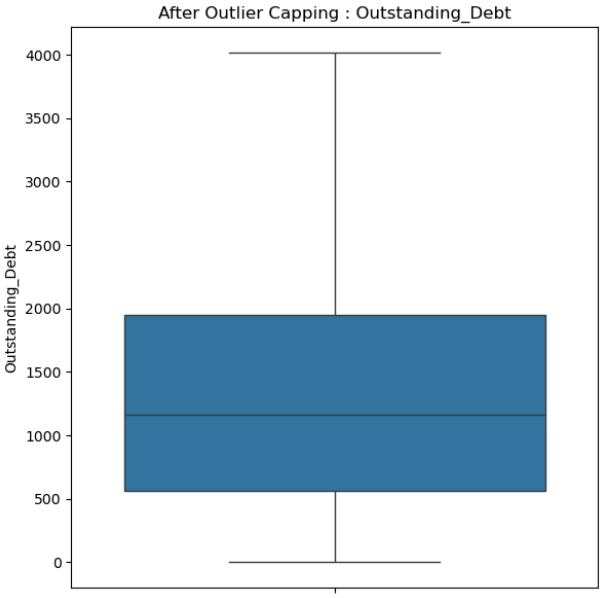

Before Outlier Capping : Changed_Credit_Limit
Changed_Credit_Limit
30
20
10
0

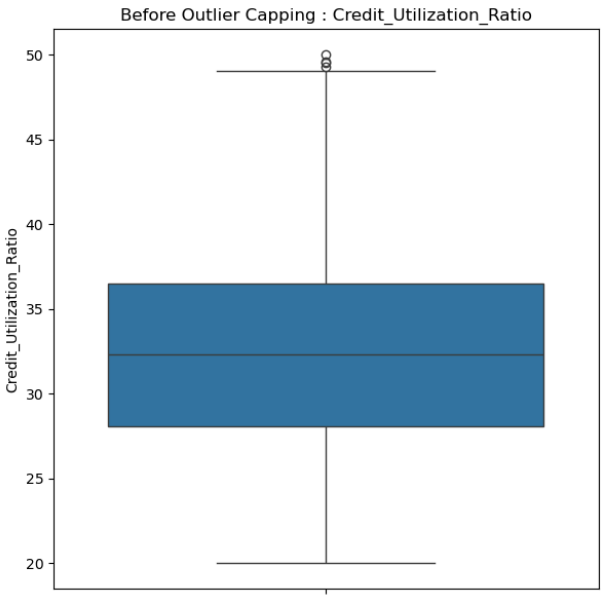

After Outlier Capping : Changed_Credit_Limit
Changed_Credit_Limit
30
25
20
15
10
5
0
−5

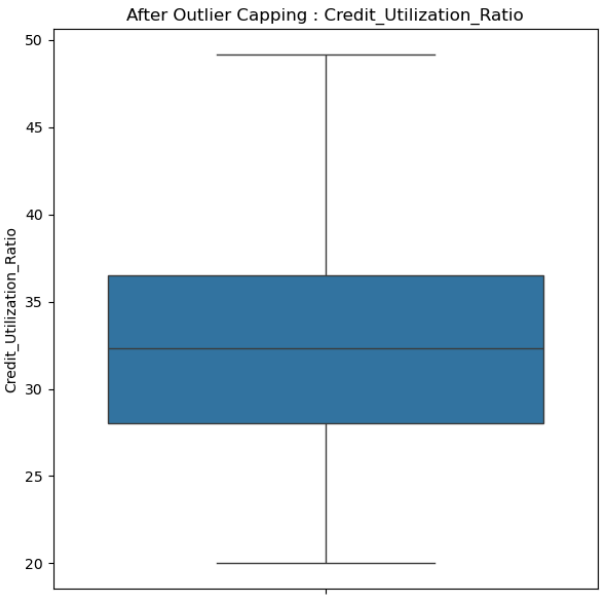

Before Outlier Capping : Num_Credit_Inquiries
Num_Credit_Inquiries
2500
2000
1500
1000
500
0

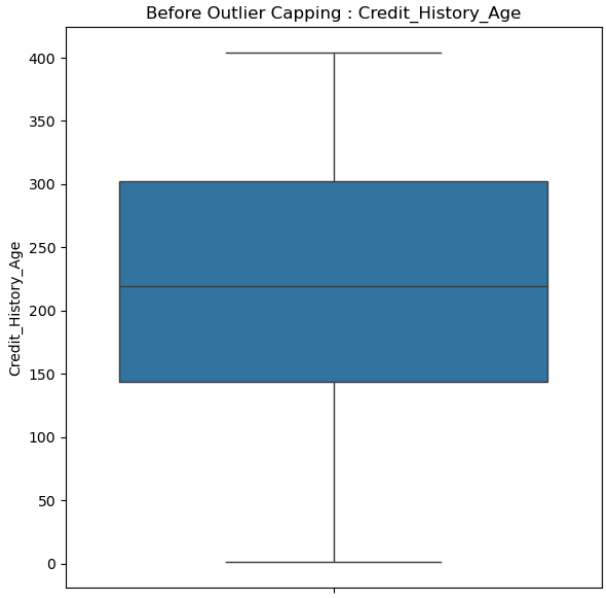

After Outlier Capping : Num_Credit_Inquiries
Num_Credit_Inquiries
17.5
15.0
12.5
10.0
7.5
5.0
2.5
0.0

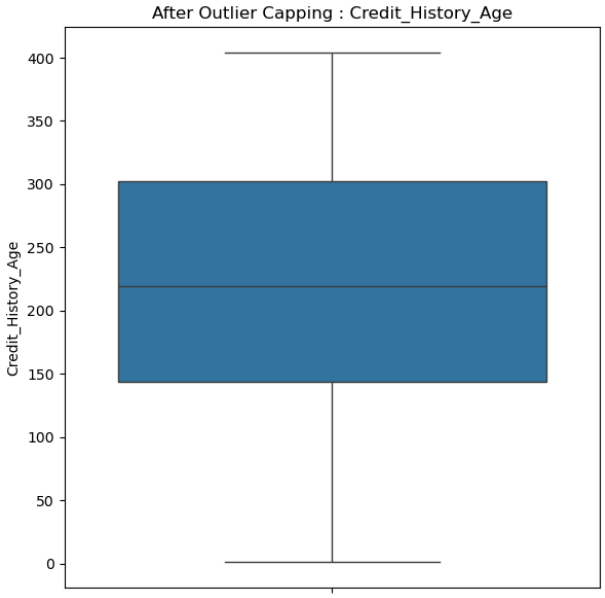

Before Outlier Capping : Outstanding_Debt
Outstanding_Debt
5000
4000
3000
2000
1000
0

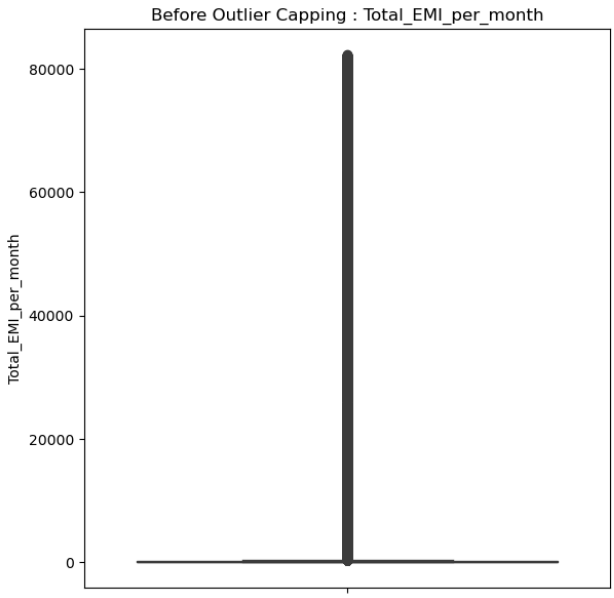

After Outlier Capping : Outstanding_Debt
Outstanding_Debt
4000
3500
3000
2500
2000
1500
1000
500
0

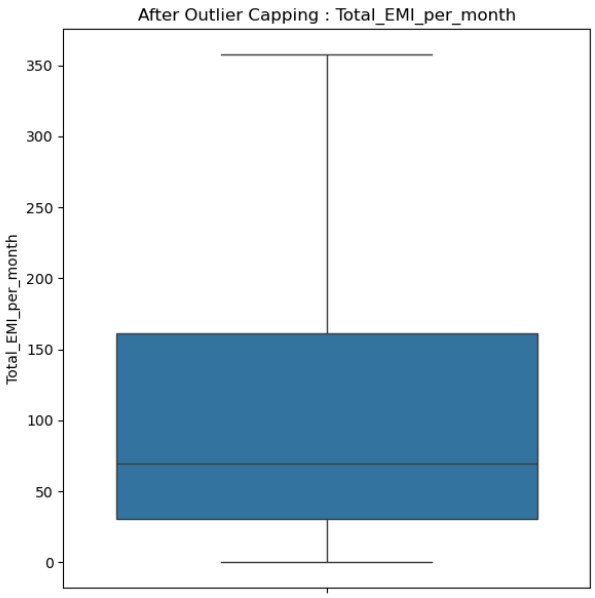

Before Outlier Capping : Credit_Utilization_Ratio
Credit_Utilization_Ratio
50
45
40
35
30
25
20

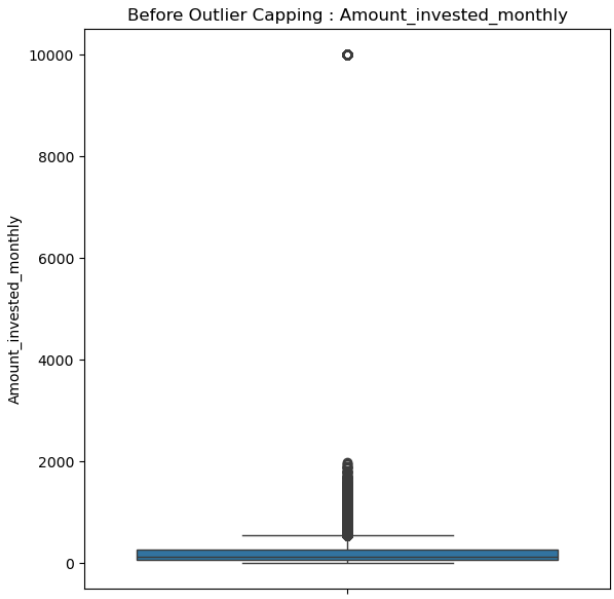

After Outlier Capping : Credit_Utilization_Ratio
Credit_Utilization_Ratio
50
45
40
35
30
25
20

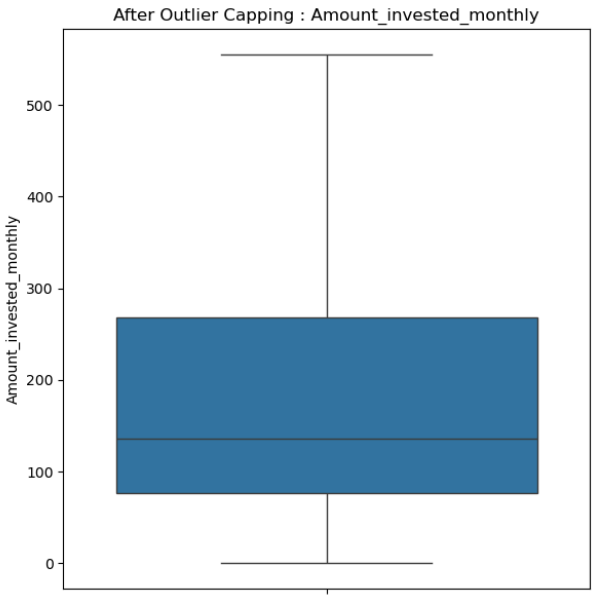

Before Outlier Capping : Credit_History_Age
Credit_History_Age
400
350
300
250
200
150
100
50
0

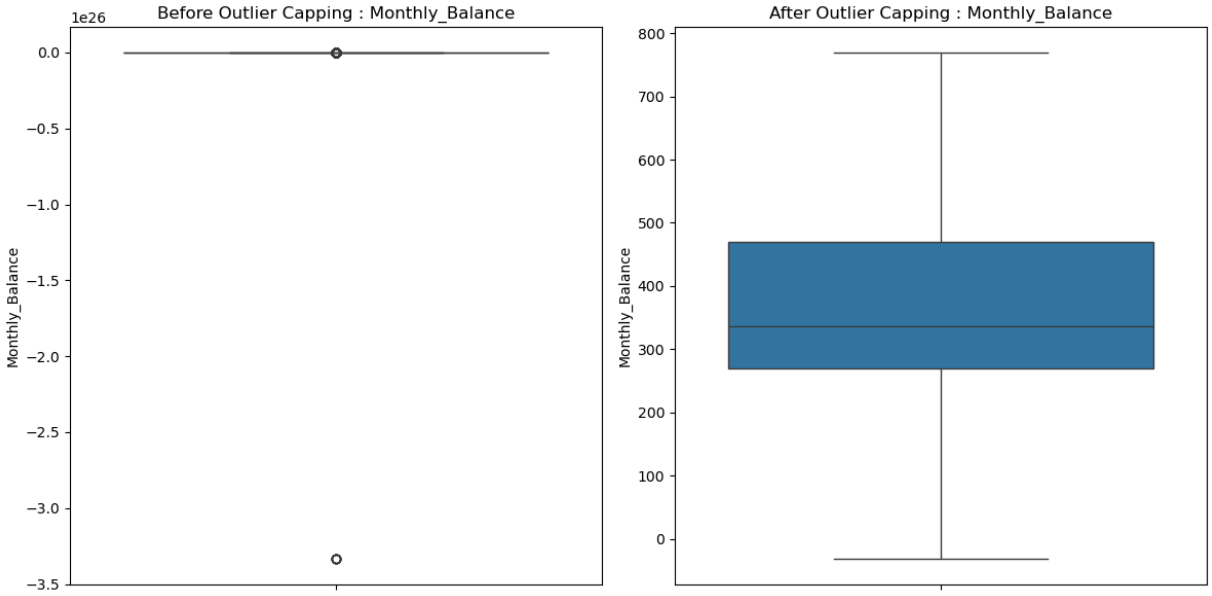

After Outlier Capping : Credit_History_Age
Credit_History_Age
400
350
300
250
200
150
100
50
0


Before Outlier Capping : Total_EMI_per_month
Total_EMI_per_month
80000
60000
40000
20000
0

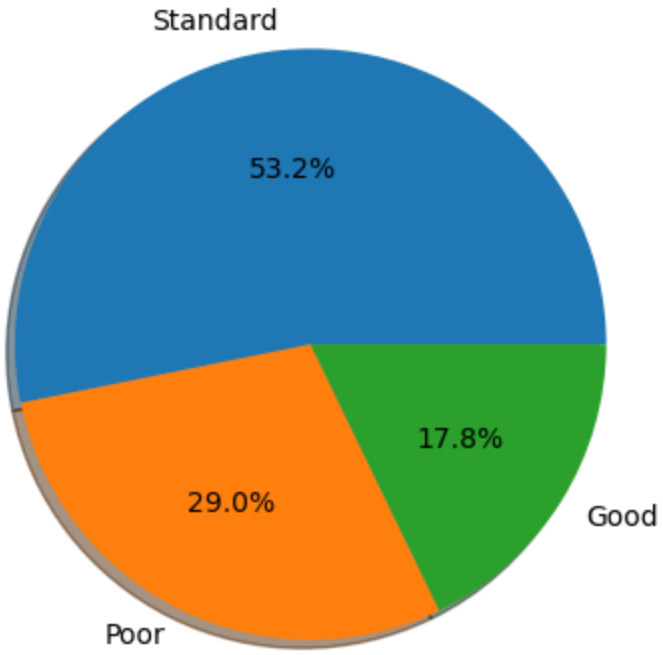

After Outlier Capping : Total_EMI_per_month
Total_EMI_per_month
350
300
250
200
150
100
50
0


Before Outlier Capping : Amount_invested_monthly
Amount_invested_monthly
10000
8000
6000
4000
2000
0


After Outlier Capping : Amount_invested_monthly
Amount_invested_monthly
500
400
300
200
100
0



In [19]:
```
credit_socre_vals = df_clean.Credit_Score.value_counts().index
credit_socre_labels = df_clean.Credit_Score.value_counts().values

plt.pie(data = df_clean,
x = credit_socre_labels,
labels = credit_socre_vals,
autopct = '%1.1f%%',
shadow = True,
radius = 1)

plt.show()
```



In [20]:
```
df_clean['Month'] = pd.to_datetime(df_clean.Month, format='%B').dt.month
```

In [21]:
```
categorical_cols = ['Occupation', 'Type_of_Loan', 'Credit_Mix', 'Payment_of_

for column in categorical_cols:
    unique_values_count = len(df_clean[column].unique())
    print(f"Number of unique values in the '{column}' column:", unique_value
```

```
Number of unique values in the 'Occupation' column: 15
Number of unique values in the 'Type_of_Loan' column: 6260
Number of unique values in the 'Credit_Mix' column: 3
Number of unique values in the 'Payment_of_Min_Amount' column: 3
Number of unique values in the 'Payment_Behaviour' column: 6
```

In [22]:
```
from sklearn.preprocessing import LabelEncoder
label_encoder = LabelEncoder()
df_clean['Type_of_Loan'] = label_encoder.fit_transform(df_clean['Type_of_Loa
```

In [23]:
```
print('Unique values in Payment_of_Min_Amount are: ', df_clean['Payment_of_M
```

```
Unique values in Payment_of_Min_Amount are:  ['No' 'NM' 'Yes']
```

In [24]:
```
target_mapping = {'No': 1, 'NM': 2, 'Yes': 3}
```

In [25]:
```
# mapping values
df_clean['Payment_of_Min_Amount'] = df_clean['Payment_of_Min_Amount'].map(ta
```

In [26]:
```
# mentioning the categorical columns where one-hot encoding needs to be perf
columns_to_encode = ['Occupation', 'Credit_Mix', 'Payment_Behaviour']

# creating dummy variables
df_dummy = pd.get_dummies(df_clean[columns_to_encode])

# concatenating the dummy variables with the original dataframe
df_processed = pd.concat([df_clean, df_dummy], axis=1)

# dropping the original categorical columns for which dummy variables were c
df_processed.drop(columns_to_encode, axis=1, inplace=True)

target_mapping = {'Poor': 1, 'Standard': 2, 'Good': 3}
df_processed['Credit_Score'] = df_processed['Credit_Score'].map(target_mappi
df_processed.dtypes[df_processed.dtypes=='object']
```

Out[26]:
```
ID              object
Customer_ID     object
Name            object
SSN             object
dtype: object
```

In [27]:
```
df_clean.drop(['ID', 'Customer_ID', 'SSN', 'Name'], axis=1, inplace=True)
```

In [28]:
```
from sklearn.ensemble import RandomForestClassifier
from sklearn.feature_selection import RFE, SelectKBest, f_classif
```

In [29]:
```
from sklearn.preprocessing import LabelEncoder
from sklearn.ensemble import RandomForestClassifier
```

```
# Assuming 'df_processed' is your cleaned dataframe and 'Credit_Score' is th
X = df_processed.drop('Credit_Score', axis=1)  # Feature set (excluding the
y = df_processed['Credit_Score']  # Target variable

# Selecting only numeric columns for training
X_numeric = X.select_dtypes(include=['float64', 'int64'])

# If you have non-numeric columns and want to encode them, apply label encod
categorical_cols = X.select_dtypes(exclude=['float64', 'int64']).columns

# Apply Label Encoding for categorical columns
for column in categorical_cols:
    le = LabelEncoder()
    X[column] = le.fit_transform(X[column].astype(str))  # Ensure strings ar

# Now we can proceed with fitting the model
rf_model = RandomForestClassifier()
rf_model.fit(X, y)
feature_importances = rf_model.feature_importances_
top_10_rf = X.columns[feature_importances.argsort()[-10:][::-1]]

# Output the top 10 features based on the RandomForest model
print("Top 10 variables from Tree-Based Method:", ', '.join(top_10_rf))
```

```
Top 10 variables from Tree-Based Method: Outstanding_Debt, Interest_Rate, Cr
edit_History_Age, Delay_from_due_date, Changed_Credit_Limit, Credit_Mix_Goo
d, ID, Credit_Mix_Standard, Num_Credit_Inquiries, Num_Credit_Card
```

In [30]:
```
from sklearn.ensemble import RandomForestClassifier
from sklearn.feature_selection import RFE

# Initialize RandomForestClassifier with parallel processing
rf_model = RandomForestClassifier(n_jobs=-1, random_state=42)

# Optimize RFE by increasing step size
rfe_selector = RFE(estimator=rf_model, n_features_to_select=10, step=5)

# Fit the RFE selector on the dataset
rfe_selector.fit(X, y)

# Extract the top 10 features
top_10_rfe = X.columns[rfe_selector.support_]

print("Top 10 variables from Method Recursive Feature Elimination:", ', '.jo
```

```
Top 10 variables from Method Recursive Feature Elimination: ID, Monthly_Inha
nd_Salary, Interest_Rate, Delay_from_due_date, Changed_Credit_Limit, Outstan
ding_Debt, Credit_Utilization_Ratio, Credit_History_Age, Amount_invested_mon
thly, Monthly_Balance
```

In [31]:
```
# Method 3: using Univariate Feature Selection
selector = SelectKBest(score_func=f_classif, k=10)
selector.fit(X, y)
top_10_univariate = X.columns[selector.get_support()]
print("Top 10 variables from Univariate Feature Selection:", ', '.join(top_1
```

```
Top 10 variables from Univariate Feature Selection: Interest_Rate, Delay_fro
m_due_date, Num_of_Delayed_Payment, Num_Credit_Inquiries, Outstanding_Debt,
Credit_History_Age, Payment_of_Min_Amount, Credit_Mix_Bad, Credit_Mix_Good,
Credit_Mix_Standard
```

In [32]:
```
imp_columns = list(set(top_10_rf.tolist() + top_10_rfe.tolist() + top_10_uni
imp_columns
```

Out[32]:
```
['Credit_History_Age',
 'Credit_Mix_Good',
 'Num_of_Delayed_Payment',
 'Monthly_Inhand_Salary',
 'Outstanding_Debt',
 'Num_Credit_Card',
 'Amount_invested_monthly',
 'Monthly_Balance',
 'Changed_Credit_Limit',
 'Num_Credit_Inquiries',
 'Credit_Utilization_Ratio',
 'Interest_Rate',
 'Delay_from_due_date',
 'Payment_of_Min_Amount',
 'Credit_Mix_Standard',
 'ID',
 'Credit_Mix_Bad']
```

In [33]:
```
print('Number of selected columns are: ', len(imp_columns))
```

```
Number of selected columns are:  17
```

In [34]:
```
import numpy as np
import pandas as pd

# Assuming df_processed is the cleaned dataframe and imp_columns are the sel
X_selected = df_processed[imp_columns]  # Define X_selected with the appropr

# Ensure only numeric columns are used for correlation
X_selected_numeric = X_selected.select_dtypes(include=[np.number])

# Check for any convertible columns (optional)
for column in X_selected.columns:
    if column not in X_selected_numeric.columns:
        try:
            X_selected_numeric[column] = pd.to_numeric(X_selected[column], e
        except:
            print(f"Could not convert column {column} to numeric. It will be

# Creating correlation matrix with numeric columns only
correlation_matrix = X_selected_numeric.corr()

# Finding highly correlated feature pairs
highly_correlated_pairs = (correlation_matrix.abs() > 0.7) & (correlation_ma

print("Highly correlated pairs of variables and their correlation values:\n"
checked_pairs = set()  # To keep track of checked pairs

for col1 in X_selected_numeric.columns:
```

```
    for col2 in X_selected_numeric.columns:
        if col1 != col2 and (col1, col2) not in checked_pairs and (col2, col
            if highly_correlated_pairs.loc[col1, col2]:
                correlation_value = correlation_matrix.loc[col1, col2]
                print(f"{col1} - {col2}: {correlation_value:.2f}")
                checked_pairs.add((col1, col2))
```

```
Highly correlated pairs of variables and their correlation values:

Outstanding_Debt - Credit_Mix_Bad: 0.77
Payment_of_Min_Amount - Credit_Mix_Good: -0.75
```

In [35]:
```
X_selected.drop(['Credit_Mix_Good', 'Credit_Mix_Bad'], axis=1, inplace=True)
```

```
C:\Users\ramak\AppData\Local\Temp\ipykernel_22864\3480153545.py:1: SettingWi
thCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/
stable/user_guide/indexing.html#returning-a-view-versus-a-copy
  X_selected.drop(['Credit_Mix_Good', 'Credit_Mix_Bad'], axis=1, inplace=Tru
e)
```

In [36]:
```
# Assuming df_processed is your main DataFrame containing the selected featu
X_selected = df_processed[imp_columns]  # imp_columns should be the list of

# Ensure X_selected is numeric
X_selected_numeric = X_selected.select_dtypes(include=['number'])

# Handle missing values
from sklearn.impute import SimpleImputer
imputer = SimpleImputer(strategy='mean')  # You can use 'median' or 'most_fr
X_selected_cleaned = pd.DataFrame(imputer.fit_transform(X_selected_numeric),

# Calculating Variance Inflation Factor (VIF)
from statsmodels.stats.outliers_influence import variance_inflation_factor
vif_df = pd.DataFrame()
vif_df["Variable"] = X_selected_cleaned.columns
vif_df["VIF"] = [variance_inflation_factor(X_selected_cleaned.values, i) for

print("Variance Inflation Factors:")
print(vif_df)
```

```
Variance Inflation Factors:
                    Variable        VIF
0          Credit_History_Age   8.984482
1      Num_of_Delayed_Payment   9.072235
2       Monthly_Inhand_Salary  10.026324
3            Outstanding_Debt   6.362409
4             Num_Credit_Card   9.869765
5     Amount_invested_monthly   4.831505
6             Monthly_Balance  16.280889
7        Changed_Credit_Limit   4.787326
8        Num_Credit_Inquiries   5.730655
9    Credit_Utilization_Ratio  25.805233
10              Interest_Rate   7.281044
11       Delay_from_due_date   5.899710
12     Payment_of_Min_Amount  12.806328
```

In [37]:
```
print('Number of selected columns are: ', len(X_selected.columns))
```

```
Number of selected columns are:  17
```

In [38]:
```
final_X_cols = X_selected.columns.tolist()
print("Final selected predictors are: ", ', '.join(final_X_cols))
```

```
Final selected predictors are:  Credit_History_Age, Credit_Mix_Good, Num_of_
Delayed_Payment, Monthly_Inhand_Salary, Outstanding_Debt, Num_Credit_Card, A
mount_invested_monthly, Monthly_Balance, Changed_Credit_Limit, Num_Credit_In
quiries, Credit_Utilization_Ratio, Interest_Rate, Delay_from_due_date, Payme
nt_of_Min_Amount, Credit_Mix_Standard, ID, Credit_Mix_Bad
```

In [39]:
```
df_final = df_processed[final_X_cols + ['Credit_Score']]
```

In [40]:
```
import pandas as pd
from sklearn.preprocessing import MinMaxScaler

# Check for non-numeric columns
non_numeric_cols = X.select_dtypes(include=['object', 'category']).columns
print("Non-numeric columns:", non_numeric_cols)

# Convert non-numeric columns to numeric (example: label encoding)
X[non_numeric_cols] = X[non_numeric_cols].apply(lambda col: pd.factorize(col

# Initialize MinMaxScaler
scaler = MinMaxScaler()

# Fit and transform the independent features
X_scaled = scaler.fit_transform(X)

print("Scaled features:")
print(X_scaled)
```

```
Non-numeric columns: Index([], dtype='object')
Scaled features:
[[0.16406164 0.98567885 0.         ... 0.         0.         0.        ]
 [0.16417164 0.98567885 0.14285714 ... 1.         0.         0.        ]
 [0.16428164 0.98567885 0.28571429 ... 0.         1.         0.        ]
 ...
 [0.62790628 0.70669654 0.71428571 ... 0.         0.         0.        ]
 [0.62791628 0.70669654 0.85714286 ... 1.         0.         0.        ]
 [0.62792628 0.70669654 1.         ... 0.         0.         0.        ]]
```

In [41]:
```
y.value_counts()
```

Out[41]:
```
Credit_Score
2    53174
1    28998
3    17828
Name: count, dtype: int64
```

In [42]:
```
len(X)
```

Out[42]:
```
100000
```

In [43]:
```
# importing SMOTE library
from imblearn.over_sampling import SMOTE

# initializing SMOTE
smote = SMOTE()

# fitting SMOTE to the data
X_bal, y_bal = smote.fit_resample(X, y)
```

In [44]:
```
y_bal.value_counts()
```

Out[44]:
```
Credit_Score
3    53174
2    53174
1    53174
Name: count, dtype: int64
```

In [45]:
```
len(X_bal)
```

Out[45]:
```
159522
```

In [46]:
```
# importing required library
from sklearn.model_selection import train_test_split

# splitting data into train and test
X_train, X_test, y_train, y_test = train_test_split(X_bal, y_bal, test_size

# finding number of rows and column of the train and test dataset
print(X_train.shape)
print(X_test.shape)
print(y_train.shape)
print(y_test.shape)
```

```
(111665, 48)
(47857, 48)
(111665,)
(47857,)
```

In [47]:
```python
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.naive_bayes import GaussianNB
from sklearn.neighbors import KNeighborsClassifier
from sklearn.model_selection import cross_validate
from sklearn.metrics import precision_score, recall_score, accuracy_score
import numpy as np

# List of classifiers to test
classifiers = [
    ('Random Forest', RandomForestClassifier())
]

# Iterate over each classifier and evaluate performance
for clf_name, clf in classifiers:
    # Perform cross-validation for all metrics at once
    scoring = ['accuracy', 'precision_macro', 'recall_macro']
    results = cross_validate(clf, X_train, y_train, cv=5, scoring=scoring, r

    # Calculate average performance for accuracy, precision, and recall
    avg_accuracy = results['test_accuracy'].mean()
    avg_precision = results['test_precision_macro'].mean()
    avg_recall = results['test_recall_macro'].mean()

    # Print the performance metrics
    print(f'Classifier: {clf_name}')
    print(f'Average Accuracy: {avg_accuracy:.4f}')
    print(f'Average Precision: {avg_precision:.4f}')
    print(f'Average Recall: {avg_recall:.4f}')
    print('-----------------------')
```

```
Classifier: Random Forest
Average Accuracy: 0.8548
Average Precision: 0.8558
Average Recall: 0.8548
-----------------------
```

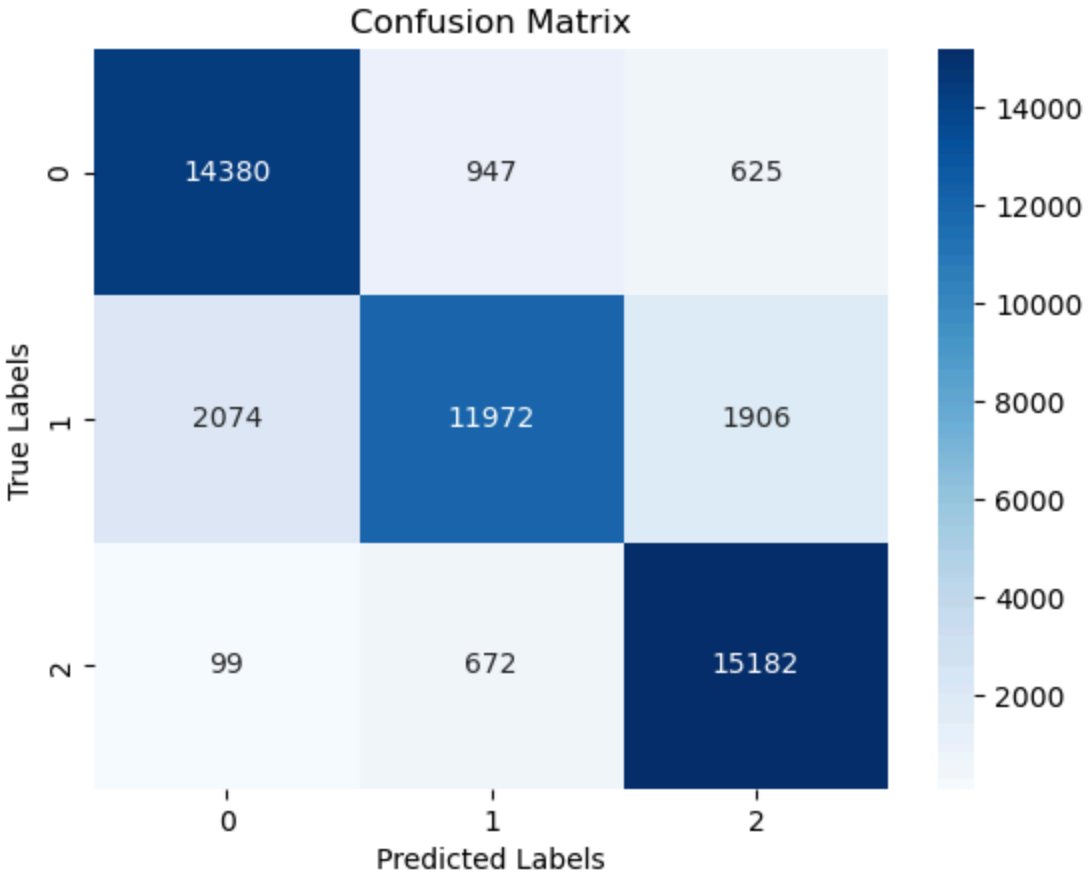
In [48]:
```python
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.naive_bayes import GaussianNB
from sklearn.neighbors import KNeighborsClassifier
from sklearn.model_selection import cross_validate
from sklearn.metrics import precision_score, recall_score, accuracy_score
import numpy as np

# List of classifiers to test
classifiers = [
    ('Decision Tree', DecisionTreeClassifier())
]

# Iterate over each classifier and evaluate performance
```

```
for clf_name, clf in classifiers:
    # Perform cross-validation for all metrics at once
    scoring = ['accuracy', 'precision_macro', 'recall_macro']
    results = cross_validate(clf, X_train, y_train, cv=5, scoring=scoring, r

    # Calculate average performance for accuracy, precision, and recall
    avg_accuracy = results['test_accuracy'].mean()
    avg_precision = results['test_precision_macro'].mean()
    avg_recall = results['test_recall_macro'].mean()

    # Print the performance metrics
    print(f'Classifier: {clf_name}')
    print(f'Average Accuracy: {avg_accuracy:.4f}')
    print(f'Average Precision: {avg_precision:.4f}')
    print(f'Average Recall: {avg_recall:.4f}')
    print('-----------------------')
```

```
Classifier: Decision Tree
Average Accuracy: 0.7741
Average Precision: 0.7736
Average Recall: 0.7741
-----------------------
```

In [49]:

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import confusion_matrix
import seaborn as sns
import matplotlib.pyplot as plt

# Initialize the classifier
rf_classifier = RandomForestClassifier(random_state=42)

# Train the classifier
rf_classifier.fit(X_train, y_train)

# Now proceed with predictions
y_test_pred = rf_classifier.predict(X_test)

# Evaluate model through a confusion matrix
cm = confusion_matrix(y_test, y_test_pred)
sns.heatmap(cm, annot=True, cmap='Blues', fmt='.0f')

plt.xlabel('Predicted Labels')
plt.ylabel('True Labels')
plt.title('Confusion Matrix')
plt.show()
```



In [50]:
```
#now we are apply this same data using liquid neural network
```

In [51]:
```
df_final
```

Out[51]:

	Credit_History_Age	Credit_Mix_Good	Num_of_Delayed_Payment	Month
0	265.0	True	7.0	
1	265.0	True	4.0	
2	267.0	True	7.0	
3	268.0	True	4.0	
4	269.0	True	4.0	
...	...	...	...	
99995	378.0	True	7.0	
99996	379.0	True	7.0	
99997	380.0	True	6.0	
99998	381.0	True	6.0	
99999	382.0	True	6.0	

100000 rows × 18 columns

In [52]:
```
df2 = df_final
```

In [53]:
```
import pandas as pd

# Assuming your original DataFrame is called df
df2 = df_final.iloc[:50000]  # Select the first 5000 rows
print(df2.shape)
```

```
(50000, 18)
```

In [54]:
```
df2
```

Out[54]:

	Credit_History_Age	Credit_Mix_Good	Num_of_Delayed_Payment	Month
0	265.0	True	7.0	
1	265.0	True	4.0	
2	267.0	True	7.0	
3	268.0	True	4.0	
4	269.0	True	4.0	
...	...	...	...	
49995	226.0	False	14.0	
49996	227.0	False	16.0	
49997	228.0	False	14.0	
49998	229.0	False	15.0	
49999	230.0	False	13.0	

50000 rows × 18 columns

In [55]:
```
df2.head(5)
```

Out[55]:

	Credit_History_Age	Credit_Mix_Good	Num_of_Delayed_Payment	Monthly_In
0	265.0	True	7.0	
1	265.0	True	4.0	
2	267.0	True	7.0	
3	268.0	True	4.0	
4	269.0	True	4.0	

In [56]:
```
df2.drop('ID',axis=1)
```

Out[56]:

	Credit_History_Age	Credit_Mix_Good	Num_of_Delayed_Payment	Month
0	265.0	True	7.0	
1	265.0	True	4.0	
2	267.0	True	7.0	
3	268.0	True	4.0	
4	269.0	True	4.0	
...	...	...	...	
49995	226.0	False	14.0	
49996	227.0	False	16.0	
49997	228.0	False	14.0	
49998	229.0	False	15.0	
49999	230.0	False	13.0	

50000 rows × 17 columns

In [57]:
```
df2 = df2[['Credit_Score','Changed_Credit_Limit']]

df2
```

Out[57]:

	Credit_Score	Changed_Credit_Limit
0	3	11.27
1	3	11.27
2	3	11.27
3	3	6.27
4	3	11.27
...	...	...
49995	1	18.19
49996	1	15.19
49997	1	18.19
49998	1	18.19
49999	1	18.19

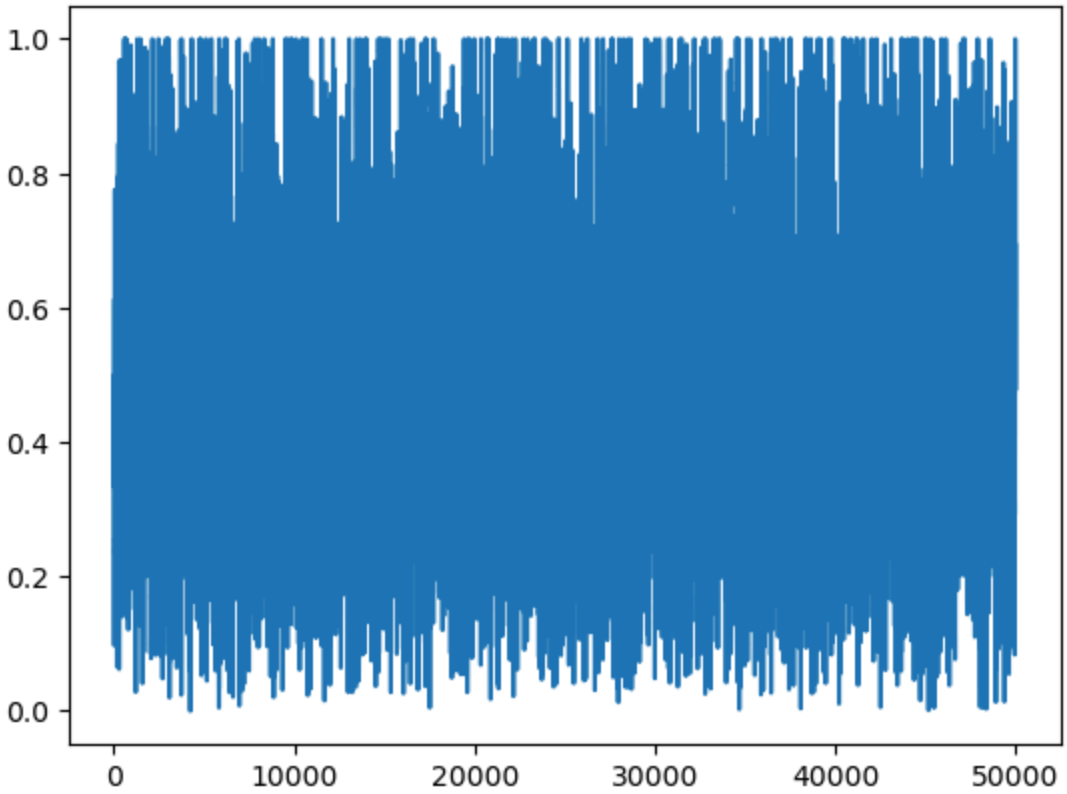
50000 rows × 2 columns

In [58]:
```
from sklearn.preprocessing import MinMaxScaler
scaler = MinMaxScaler(feature_range=(0,1))
scaled = scaler.fit_transform(np.array(df['Changed_Credit_Limit']).reshape(-
df2['Changed_Credit_Limit'] = scaler.fit_transform(np.array(df2['Changed_Cre
```

```
C:\Users\ramak\AppData\Local\Temp\ipykernel_22864\1759463014.py:4: SettingWi
thCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/
stable/user_guide/indexing.html#returning-a-view-versus-a-copy
  df2['Changed_Credit_Limit'] = scaler.fit_transform(np.array(df2['Changed_C
redit_Limit']).reshape(-1,1))
```

In [59]:
```
plt.plot(df2.index,df2['Changed_Credit_Limit'])
```

Out[59]:
```
[<matplotlib.lines.Line2D at 0x1ef516fcf70>]
```



In [60]:
```
def create_sequences(data,step_size):
    sequences = []
    x,y=[],[]

    for i in range(len(data) - step_size):
        x = data.iloc[i:i+step_size,:].values
        y = data.iloc[i+step_size,:].values

        sequences.append((x,y))

    return pd.DataFrame(sequences)
```

In [61]:
```
sequences = create_sequences(df2,60)
sequences
```

Out[61]:

	0	1
0	[[3.0, 0.49866554291333054], [3.0, 0.498665542...	[2.0, 0.6632954066582385]
1	[[3.0, 0.49866554291333054], [3.0, 0.498665542...	[2.0, 0.6913892400618065]
2	[[3.0, 0.49866554291333054], [3.0, 0.358196375...	[2.0, 0.6632954066582385]
3	[[3.0, 0.3581963758954909], [3.0, 0.4986655429...	[1.0, 0.6632954066582385]
4	[[3.0, 0.49866554291333054], [3.0, 0.442477876...	[3.0, 0.4135412277005197]
...	...	...
49935	[[1.0, 0.6866132883831999], [2.0, 0.2803764573...	[1.0, 0.6930748700660205]
49936	[[2.0, 0.2803764573676078], [2.0, 0.2803764573...	[1.0, 0.6087933698553167]
49937	[[2.0, 0.2803764573676078], [2.0, 0.2803764573...	[1.0, 0.6930748700660205]
49938	[[2.0, 0.2803764573676078], [2.0, 0.2803764573...	[1.0, 0.6930748700660205]
49939	[[2.0, 0.2803764573676078], [2.0, 0.1960949571...	[1.0, 0.6930748700660205]

49940 rows × 2 columns

In [62]:
```
df3 = pd.DataFrame(columns = [i for i in range(61)])
for i in range(len(sequences)):
    li = []
    for j in range(60):
        num = sequences.iloc[i,0][j][0]
        li.append(num)
    li.append(sequences.iloc[i,1][0])

    df3.loc[len(df3)] = li
```

In [63]:
```
df4 = df3
df3 = df3.set_index(df2.index[60:])
```

In [64]:
```
df4['Credit_Score'] = df2.index[60:]
```

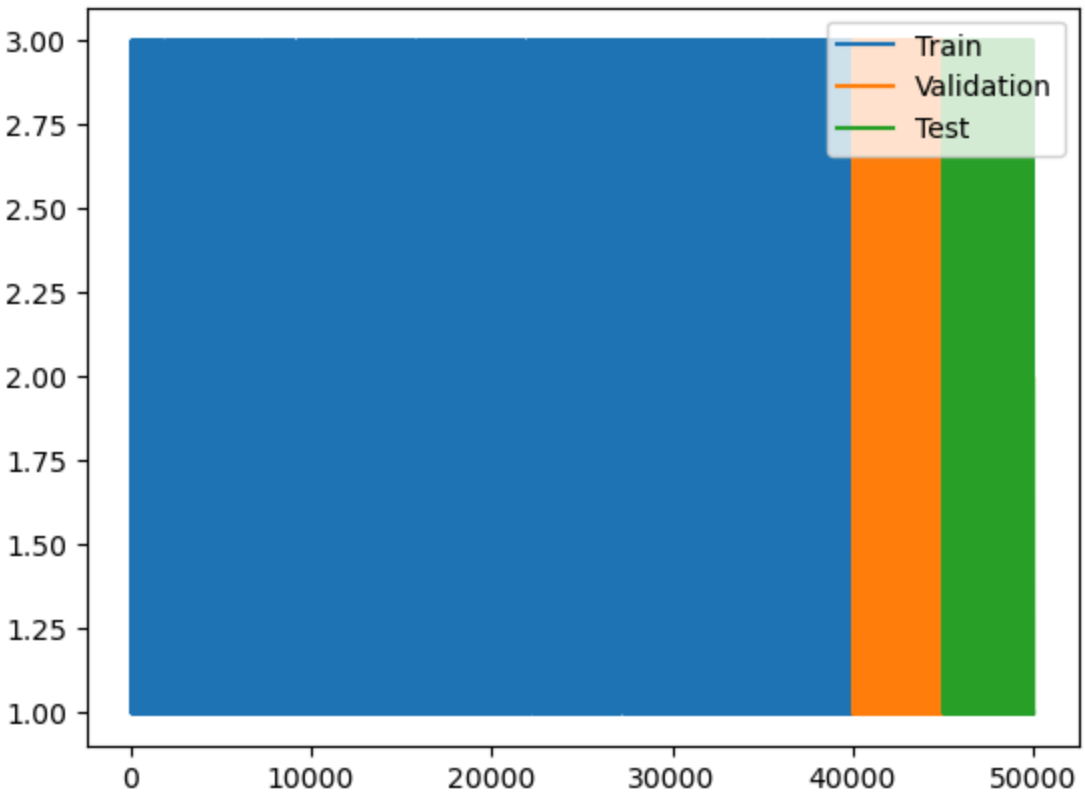
In [65]:
```
def windowed_df_to_date_X_y(windowed_dataframe):
    df_as_np = windowed_dataframe.to_numpy()
    Credit_Score = df_as_np[:, -1]
    middle_matrix = df_as_np[:, 0:-2]
    X = middle_matrix.reshape((len(Credit_Score), middle_matrix.shape[1], 1))
    Y = df_as_np[:, -2]
    return Credit_Score, X.astype(np.float32), Y.astype(np.float32)
```

```
Credit_Score, X, y = windowed_df_to_date_X_y(df4)

Credit_Score.shape, X.shape, y.shape
```

Out[65]: ((49940,), (49940, 60, 1), (49940,))

In [66]:
```
q_80 = int(len(Credit_Score) * .8)
q_90 = int(len(Credit_Score) * .9)

Credit_Score_train, X_train, y_train = Credit_Score[:q_80], X[:q_80], y[:q_8

Credit_Score_val, X_val, y_val = Credit_Score[q_80:q_90], X[q_80:q_90], y[q_
Credit_Score_test, X_test, y_test = Credit_Score[q_90:], X[q_90:], y[q_90:]

plt.plot(Credit_Score_train, y_train)
plt.plot(Credit_Score_val, y_val)
plt.plot(Credit_Score_test, y_test)

plt.legend(['Train', 'Validation', 'Test'])
```

Out[66]: <matplotlib.legend.Legend at 0x1ef521d71c0>



In [67]:
```
import tensorflow as tf
from tensorflow import keras
import numpy as np

# Define the Liquid Time-Constant (LTC) cell
class LTCCell(keras.layers.Layer):
    def __init__(self, units, **kwargs):
        super(LTCCell, self).__init__(**kwargs)
        self.units = units
```

```
        self.state_size = units  # Define state size

    def build(self, input_shape):
        self.W = self.add_weight(shape=(input_shape[-1], self.units), initia
        self.U = self.add_weight(shape=(self.units, self.units), initializer
        self.b = self.add_weight(shape=(self.units,), initializer='zeros', t

    def call(self, inputs, states):
        prev_output = states[0]
        time_constant = tf.nn.sigmoid(tf.matmul(inputs, self.W) + tf.matmul(
        new_output = prev_output + time_constant * (inputs - prev_output)
        return new_output, [new_output]

# Create a simple LTC model
def create_ltc_model(input_shape, units):
    inputs = keras.Input(shape=input_shape)
    ltc = keras.layers.RNN(LTCCell(units))(inputs)
    outputs = keras.layers.Dense(1)(ltc)
    model = keras.Model(inputs=inputs, outputs=outputs)
    return model
```

In [68]:

```
model = create_ltc_model(input_shape=(60, 1), units=10)
model.compile(optimizer='adam', loss='mse')

# Train the model
history = model.fit(X_train, y_train, validation_data = (X_val,y_val) ,epoch

# Make predictions
predictions = model.predict(X_train)
```

```
Epoch 1/100
1249/1249 [==============================] - 24s 16ms/step - loss: 0.2438 -
val_loss: 0.2265
Epoch 2/100
1249/1249 [==============================] - 18s 15ms/step - loss: 0.2266 -
val_loss: 0.2247
Epoch 3/100
1249/1249 [==============================] - 18s 14ms/step - loss: 0.2239 -
val_loss: 0.2246
Epoch 8/100
1249/1249 [==============================] - 18s 14ms/step - loss: 0.2236 -
val_loss: 0.2277
Epoch 9/100
1249/1249 [==============================] - 18s 15ms/step - loss: 0.2234 -
val_loss: 0.2234
Epoch 10/100
1249/1249 [==============================] - 17s 14ms/step - loss: 0.2231 -
val_loss: 0.2227
Epoch 11/100
1249/1249 [==============================] - 17s 14ms/step - loss: 0.2224 -
val_loss: 0.2226
Epoch 12/100
1249/1249 [==============================] - 18s 14ms/step - loss: 0.2222 -
val_loss: 0.2224
Epoch 13/100
1249/1249 [==============================] - 18s 14ms/step - loss: 0.2219 -
val_loss: 0.2216
Epoch 14/100
1249/1249 [==============================] - 17s 14ms/step - loss: 0.2219 -
val_loss: 0.2218
Epoch 15/100
1249/1249 [==============================] - 17s 14ms/step - loss: 0.2213 -
val_loss: 0.2213
Epoch 16/100
 948/1249 [=====================>........] - ETA: 3s - loss: 0.2207
```

```
IOPub message rate exceeded.
The Jupyter server will temporarily stop sending output
to the client in order to avoid crashing it.
To change this limit, set the config variable
`--ServerApp.iopub_msg_rate_limit`.

Current values:
ServerApp.iopub_msg_rate_limit=1000.0 (msgs/sec)
ServerApp.rate_limit_window=3.0 (secs)
```

```
1249/1249 [==============================] - 18s 15ms/step - loss: 0.2184 -
val_loss: 0.2199
Epoch 23/100
1249/1249 [==============================] - 18s 15ms/step - loss: 0.2186 -
val_loss: 0.2184
Epoch 24/100
1249/1249 [==============================] - 19s 15ms/step - loss: 0.2182 -
val_loss: 0.2183
Epoch 25/100
1249/1249 [==============================] - 17s 14ms/step - loss: 0.2176 -
val_loss: 0.2180
Epoch 26/100
1249/1249 [==============================] - 18s 14ms/step - loss: 0.2166 -
val_loss: 0.2167
Epoch 30/100
1249/1249 [==============================] - 18s 15ms/step - loss: 0.2162 -
val_loss: 0.2171
Epoch 31/100
1249/1249 [==============================] - 18s 14ms/step - loss: 0.2158 -
val_loss: 0.2161
Epoch 32/100
1249/1249 [==============================] - 18s 14ms/step - loss: 0.2156 -
val_loss: 0.2163
Epoch 33/100
1249/1249 [==============================] - 17s 14ms/step - loss: 0.2150 -
val_loss: 0.2157
Epoch 34/100
1249/1249 [==============================] - 18s 14ms/step - loss: 0.2146 -
val_loss: 0.2157
Epoch 35/100
1249/1249 [==============================] - 18s 14ms/step - loss: 0.2144 -
val_loss: 0.2149
Epoch 36/100
1249/1249 [==============================] - 18s 14ms/step - loss: 0.2140 -
val_loss: 0.2165
Epoch 37/100
1249/1249 [==============================] - 17s 14ms/step - loss: 0.2136 -
val_loss: 0.2149
Epoch 38/100
1249/1249 [==============================] - 17s 14ms/step - loss: 0.2132 -
val_loss: 0.2136
Epoch 39/100
 327/1249 [======>.......................] - ETA: 14s - loss: 0.2136
IOPub message rate exceeded.
The Jupyter server will temporarily stop sending output
to the client in order to avoid crashing it.
To change this limit, set the config variable
`--ServerApp.iopub_msg_rate_limit`.

Current values:
ServerApp.iopub_msg_rate_limit=1000.0 (msgs/sec)
ServerApp.rate_limit_window=3.0 (secs)
```

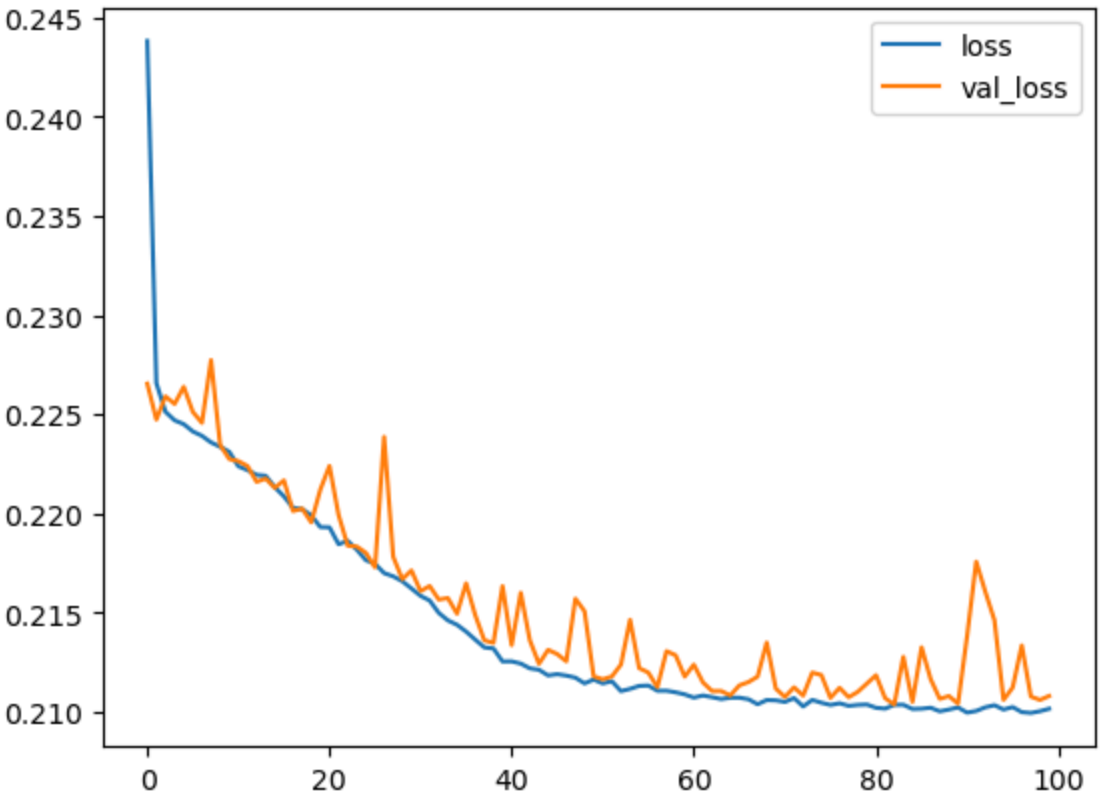
```
1249/1249 [==============================] - 19s 15ms/step - loss: 0.2125 -
val_loss: 0.2163
Epoch 41/100
1249/1249 [==============================] - 17s 14ms/step - loss: 0.2125 -
val_loss: 0.2134
Epoch 42/100
1249/1249 [==============================] - 18s 14ms/step - loss: 0.2124 -
val_loss: 0.2160
Epoch 43/100
1249/1249 [==============================] - 23s 18ms/step - loss: 0.2122 -
val_loss: 0.2136
Epoch 44/100
1249/1249 [==============================] - 33s 26ms/step - loss: 0.2121 -
val_loss: 0.2124
Epoch 45/100
1249/1249 [==============================] - 33s 26ms/step - loss: 0.2118 -
val_loss: 0.2131
Epoch 46/100
1249/1249 [==============================] - 32s 26ms/step - loss: 0.2119 -
val_loss: 0.2129
Epoch 47/100
1249/1249 [==============================] - 33s 26ms/step - loss: 0.2118 -
val_loss: 0.2125
Epoch 48/100
1249/1249 [==============================] - 33s 26ms/step - loss: 0.2117 -
val_loss: 0.2157
Epoch 49/100
1249/1249 [==============================] - 32s 25ms/step - loss: 0.2114 -
val_loss: 0.2151
Epoch 50/100
1249/1249 [==============================] - 33s 26ms/step - loss: 0.2116 -
val_loss: 0.2118
Epoch 51/100
1249/1249 [==============================] - 32s 26ms/step - loss: 0.2114 -
val_loss: 0.2116
Epoch 52/100
1249/1249 [==============================] - 31s 25ms/step - loss: 0.2115 -
val_loss: 0.2118
Epoch 53/100
1249/1249 [==============================] - 32s 26ms/step - loss: 0.2110 -
val_loss: 0.2124
Epoch 54/100
1249/1249 [==============================] - 32s 26ms/step - loss: 0.2112 -
val_loss: 0.2146
Epoch 55/100
1249/1249 [==============================] - 35s 28ms/step - loss: 0.2113 -
val_loss: 0.2122
Epoch 56/100
1249/1249 [==============================] - 33s 26ms/step - loss: 0.2113 -
val_loss: 0.2120
Epoch 57/100
1249/1249 [==============================] - 36s 29ms/step - loss: 0.2111 -
val_loss: 0.2112
Epoch 58/100
1249/1249 [==============================] - 35s 28ms/step - loss: 0.2111 -
val_loss: 0.2131
```

```
Epoch 59/100
1249/1249 [==============================] - 31s 25ms/step - loss: 0.2110 -
val_loss: 0.2128
Epoch 60/100
1249/1249 [==============================] - 33s 26ms/step - loss: 0.2109 -
val_loss: 0.2118
Epoch 61/100
1249/1249 [==============================] - 32s 25ms/step - loss: 0.2106 -
val_loss: 0.2110
Epoch 65/100
1249/1249 [==============================] - 32s 25ms/step - loss: 0.2107 -
val_loss: 0.2108
Epoch 66/100
1249/1249 [==============================] - 32s 25ms/step - loss: 0.2107 -
val_loss: 0.2113
Epoch 67/100
1249/1249 [==============================] - 32s 26ms/step - loss: 0.2106 -
val_loss: 0.2115
Epoch 68/100
1249/1249 [==============================] - 31s 25ms/step - loss: 0.2104 -
val_loss: 0.2118
Epoch 69/100
1249/1249 [==============================] - 20s 16ms/step - loss: 0.2106 -
val_loss: 0.2135
Epoch 70/100
1249/1249 [==============================] - 16s 13ms/step - loss: 0.2106 -
val_loss: 0.2112
Epoch 71/100
1249/1249 [==============================] - 16s 13ms/step - loss: 0.2105 -
val_loss: 0.2107
Epoch 72/100
1249/1249 [==============================] - 15s 12ms/step - loss: 0.2107 -
val_loss: 0.2112
Epoch 73/100
1249/1249 [==============================] - 15s 12ms/step - loss: 0.2103 -
val_loss: 0.2108
Epoch 74/100
1249/1249 [==============================] - 15s 12ms/step - loss: 0.2106 -
val_loss: 0.2120
Epoch 75/100
1249/1249 [==============================] - 15s 12ms/step - loss: 0.2105 -
val_loss: 0.2119
Epoch 76/100
1249/1249 [==============================] - 15s 12ms/step - loss: 0.2103 -
val_loss: 0.2107
Epoch 77/100
1249/1249 [==============================] - 15s 12ms/step - loss: 0.2104 -
val_loss: 0.2112
Epoch 78/100
1249/1249 [==============================] - 15s 12ms/step - loss: 0.2103 -
val_loss: 0.2107
Epoch 79/100
1249/1249 [==============================] - 15s 12ms/step - loss: 0.2103 -
val_loss: 0.2110
Epoch 80/100
1249/1249 [==============================] - 15s 12ms/step - loss: 0.2104 -
```

```
val_loss: 0.2114
Epoch 81/100
1249/1249 [==============================] - 15s 12ms/step - loss: 0.2102 -
val_loss: 0.2118
Epoch 82/100
1249/1249 [==============================] - 15s 12ms/step - loss: 0.2102 -
val_loss: 0.2107
Epoch 83/100
1249/1249 [==============================] - 15s 12ms/step - loss: 0.2103 -
val_loss: 0.2104
Epoch 84/100
1249/1249 [==============================] - 15s 12ms/step - loss: 0.2104 -
val_loss: 0.2128
Epoch 85/100
1249/1249 [==============================] - 15s 12ms/step - loss: 0.2101 -
val_loss: 0.2105
Epoch 86/100
1249/1249 [==============================] - 15s 12ms/step - loss: 0.2102 -
val_loss: 0.2133
Epoch 87/100
1249/1249 [==============================] - 15s 12ms/step - loss: 0.2102 -
val_loss: 0.2116
Epoch 88/100
1249/1249 [==============================] - 15s 12ms/step - loss: 0.2100 -
val_loss: 0.2106
Epoch 89/100
1249/1249 [==============================] - 15s 12ms/step - loss: 0.2101 -
val_loss: 0.2108
Epoch 90/100
1249/1249 [==============================] - 15s 12ms/step - loss: 0.2102 -
val_loss: 0.2104
Epoch 91/100
1249/1249 [==============================] - 15s 12ms/step - loss: 0.2100 -
val_loss: 0.2138
Epoch 92/100
1249/1249 [==============================] - 15s 12ms/step - loss: 0.2100 -
val_loss: 0.2176
Epoch 93/100
1249/1249 [==============================] - 15s 12ms/step - loss: 0.2102 -
val_loss: 0.2160
Epoch 94/100
1249/1249 [==============================] - 15s 12ms/step - loss: 0.2103 -
val_loss: 0.2146
Epoch 95/100
1249/1249 [==============================] - 15s 12ms/step - loss: 0.2101 -
val_loss: 0.2106
Epoch 96/100
1249/1249 [==============================] - 15s 12ms/step - loss: 0.2102 -
val_loss: 0.2112
Epoch 97/100
1249/1249 [==============================] - 15s 12ms/step - loss: 0.2100 -
val_loss: 0.2133
Epoch 98/100
1249/1249 [==============================] - 15s 12ms/step - loss: 0.2099 -
val_loss: 0.2108
Epoch 99/100
```

```
1249/1249 [==============================] - 15s 12ms/step - loss: 0.2100 -
val_loss: 0.2106
Epoch 100/100
1249/1249 [==============================] - 16s 13ms/step - loss: 0.2102 -
val_loss: 0.2108
1249/1249 [==============================] - 14s 5ms/step
```

In [69]:
```
plt.plot(history.history['loss'],label = 'loss')
plt.plot(history.history['val_loss'],label='val_loss')
plt.legend()
plt.show()
```



In [70]:
```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.metrics import mean_squared_error, r2_score

# Generate predictions
test_predictions = model.predict(X_test).flatten()
val_predictions = model.predict(X_val).flatten()

# Extract Credit Score feature
# Replace 0 with the correct column index for Credit Score if X_test is a Nu
Credit_Score_test = X_test[:, 0]

# Calculate and print RMSE
test_rmse = np.sqrt(mean_squared_error(y_test, test_predictions))
print("Test RMSE:", test_rmse)

# Calculate and print R-squared for training data
train_predictions = model.predict(X_train).flatten()
r2 = r2_score(y_train, train_predictions)
```

```
print(f"R-squared (Training Data): {r2}")

# Model summary (for TensorFlow/Keras models)
model.summary()
```

```
157/157 [==============================] - 3s 4ms/step
157/157 [==============================] - 1s 4ms/step
Test RMSE: 0.45002186
1249/1249 [==============================] - 4s 3ms/step
R-squared (Training Data): 0.5440285228350166
Model: "model"
_________________________________________________________________
 Layer (type)                Output Shape              Param #
=================================================================
 input_1 (InputLayer)        [(None, 60, 1)]           0

 rnn (RNN)                   (None, 10)                120

 dense (Dense)               (None, 1)                 11

=================================================================
Total params: 131
Trainable params: 131
Non-trainable params: 0
_________________________________________________________________
```

In [ ]: